MLE-World: Predicting Machine Learning Experiment Outcomes with a Learned World Model

Brian Ko[1], **Han Wang**[2], **Hejia Geng**[3], **Li Erran Li**[4], **Sherry Yang**[5]
[1]University of Washington [2]University of Illinois at Urbana-Champaign
[3]AItonomy Foundation [4]Amazon [5]New York University

Abstract

AI research agents can propose, execute, and improve programs that solve machine learning tasks. However, each step runs a program that trains and evaluates ML models, making execution time the bottleneck that prevents an agent from trying many candidate programs per step. We introduce MLE-World, a world model finetuned from Qwen3.5-4B that predicts a program's terminal output (error traces, training logs, and validation scores) from its code and data description without executing it, together with a confidence score estimating the prediction's correctness. Because a prediction costs one inference call rather than a training run, MLE-World makes test time scaling (TTS) affordable for these agents. We build five TTS policies, guided by MLE-World, that compare several candidates per step by their predicted outputs: Sequential revision, Best-of-N, Pairwise judging, Fusion, and a UCB style rule that also adds an exploration bonus from this confidence score. Since predictions can be wrong, we run a candidate for real only when its confidence score is low. On 17 MLE-bench Lite tasks, MLE-World matches the real terminal output on 77.9% of held out programs, versus 41.1% for much larger prompted models. MLE-World guided TTS achieves up to 33.6% higher observed mean performance than the agent without a world model, with less real execution. Project webpage: `https://mle-world.github.io/`.

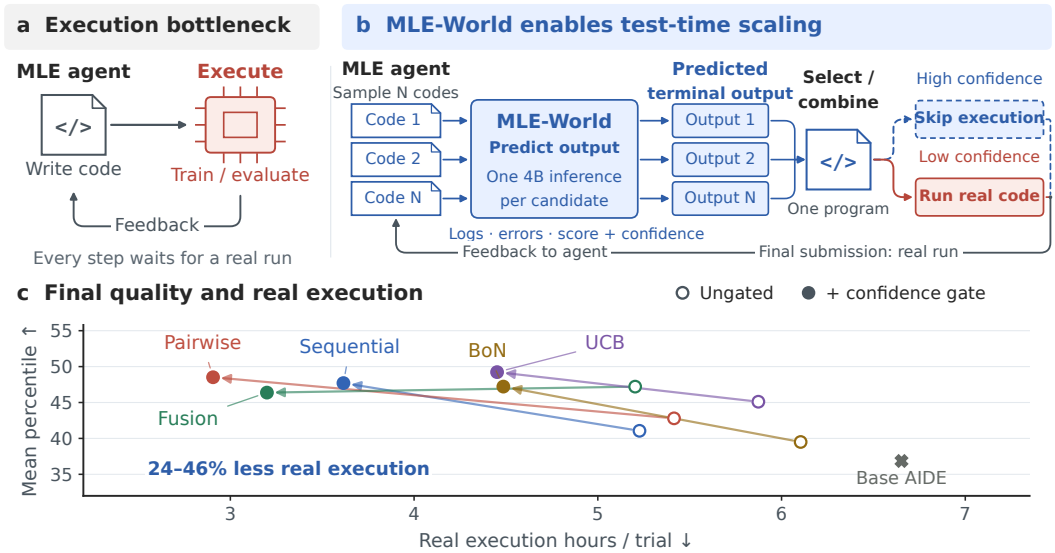


Figure 1: **Fast, cheap feedback with MLE-World.** (a) Real execution. (b) Predict terminal output per candidate, select/combine, then gate. (c) World model guided policies have higher observed mean percentiles than the agent without a world model; gating (open to filled arrows) yields higher means for four of five. Selected by best real validation; 17 tasks × three seeds (51 trials), 8 h each. The x-axis reports mean elapsed time spent on real execution across tasks.

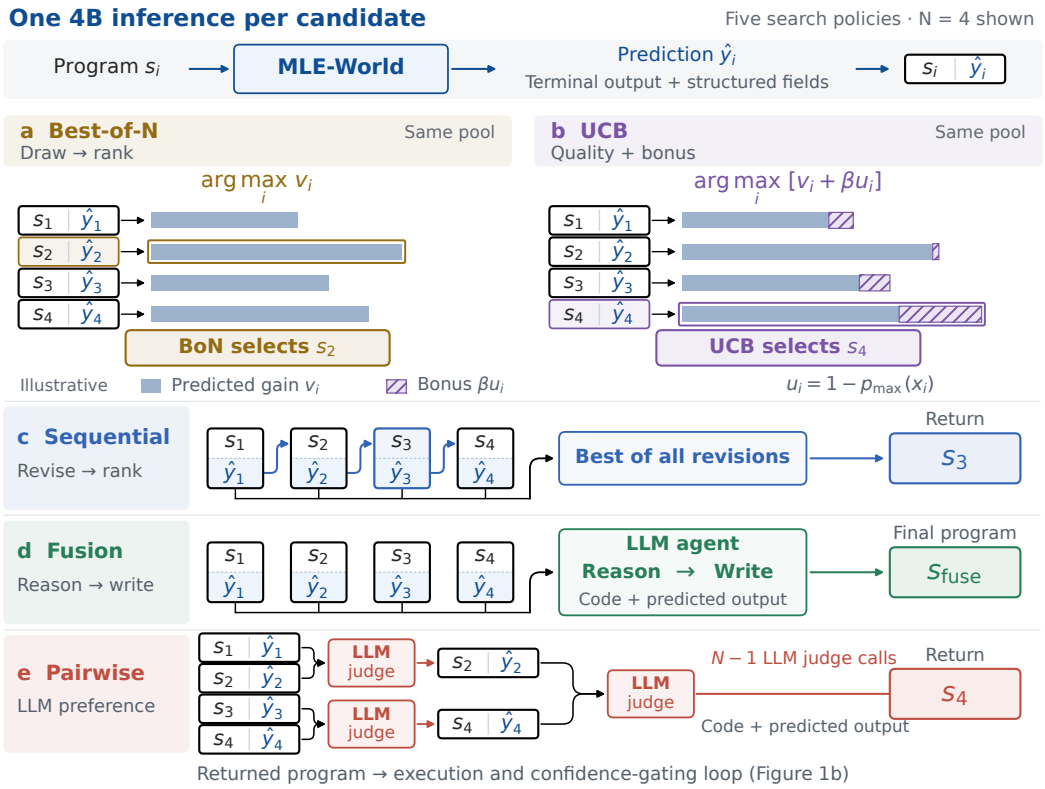


Figure 2: **Five MLE-World search policies.** Tiles show code s_i and prediction $\hat{y}_i$: terminal output plus structured fields (Equation 1). Write $x_i = x_{s_i}$, $v_i = v(s_i)$, and $u_i = u(x_i)$. UCB ranks by $v_i + \beta u_i$ (Equation 4). Illustrated with $N = 4$ eligible candidates; each policy returns one program.

1 Introduction

AI research agents repeatedly write programs, execute them, inspect terminal output, and revise solutions (Jiang et al., 2025; Nam et al., 2026), but this feedback is expensive because each step trains and evaluates a model. Therefore, execution time limits how many ideas they can test (Toledo et al., 2026; Hambardzumyan et al., 2026; Yang et al., 2026a), making cheap feedback and selective execution central to efficient search.

Prior work prunes unpromising branches (Liu et al., 2025; Toledo et al., 2026), stops training runs early (Kulibaba et al., 2025), or ranks candidates before execution (Zheng et al., 2026; Foster et al., 2026). These methods reduce the number of executions but not their cost. Every candidate that survives still needs a full training run, so execution remains the bottleneck. They also discard information, since a pruned candidate yields no error trace or training log to guide a repair.

The high cost of training makes machine learning engineering (MLE) a natural setting for world models that predict execution feedback without running the program (Yang, 2026). MLE-World predicts a candidate solution's terminal output from its code and the task's data description. We finetune Qwen3.5-4B on logged pairs of code and output from successful and failed runs. One inference call predicts error traces, training logs, and the validation score, supporting both debugging and candidate selection.

Cheap predicted feedback makes test time scaling (TTS) affordable for MLE agents. TTS can scale along two dimensions: sampling more complete trajectories (Brown et al., 2024; Snell et al., 2024) or taking more interaction steps within a trajectory (Shen et al., 2025). In MLE, both require additional training runs, limiting search under a fixed time budget. Prior agents execute every candidate (Nam et al., 2026) or rank candidates without execution feedback (Zheng et al., 2026). We focus on scaling interactions so the agent can adapt to feedback throughout the search. MLE-World makes

this affordable by letting the agent compare several candidates using predicted error traces, training logs, and scores, then execute only the selected candidate. We build five policies on these predictions: Best-of-N (BoN), Sequential revision, Pairwise judging, Fusion, and a rule based on upper confidence bounds (UCB).

MLE-World also reports a confidence score with each prediction, which serves search in two ways. First, it guides selection. Greedy selection by validation score can limit exploration (Toledo et al., 2026; Hambardzumyan et al., 2026) and overlook candidates the world model undervalues, so UCB adds an uncertainty bonus to predicted quality. Second, it decides when to trust a prediction. Wrong predictions can mislead search (Janner et al., 2019), so a confidence gate trusts confident predictions and runs the program otherwise. The gate works with any of the five policies.

Across 17 MLE-bench Lite tasks (Chan et al., 2025), MLE-World matches observed terminal output on 77.9% of held out programs versus 41.1% for the best prompted baseline, with 13–1,061$\times$ less GPU inference time. MLE-World guided search achieves up to 33.6% higher observed mean performance than the agent without a world model while reducing real execution. These results suggest that cheap feedback from a world model enables test time scaling, improving agent performance by exploring more candidate programs.

We summarize our main contributions as follows:

- **An execution world model for MLE.** We introduce MLE-World, a 4B model that reads a program's code and data description and predicts its terminal output, including crashes, training logs, and validation scores, outperforming much larger prompted models.
- **World model guided test time scaling.** We build five TTS policies that use MLE-World to select among several candidates per step by their predicted execution feedback, then run only the selected one. All five have higher observed mean percentiles than the agent without a world model.
- **Selective execution using model confidence.** We run a program for real only when MLE-World's prediction has low confidence. Gating reduces real execution and yields higher observed mean percentiles for most policies.

2 Related work

MLE agents. MLE agents built on AIDE (Jiang et al., 2025; Nam et al., 2026; Ou et al., 2025) cut execution cost by pruning branches (Liu et al., 2025; Toledo et al., 2026), extrapolating partial training runs (Kulibaba et al., 2025), or ranking candidates before execution (Foster et al., 2026; White et al., 2021). Closest to our search, ForeAgent (Zheng et al., 2026) keeps one of several Improve candidates by pairwise preference with a confidence gate and executes it. A score or preference carries no error trace or training log to guide a repair, and the chosen program still runs; MLE-World predicts its terminal output, and its confidence decides whether a program must run.

World models for LLM agents. World models predict an action's outcome without performing it (Ha & Schmidhuber, 2018; Hafner et al., 2019; Janner et al., 2019). LLM world models, prompted (Thimmaiah et al., 2025; Wang et al., 2024; Gu et al., 2024) or trained on agent logs (Chae et al., 2025; Fang et al., 2025; Yu et al., 2025; Guan et al., 2026; Sun et al., 2026; Zuo et al., 2026), serve web, computer use, and software agents. Closest to ours, DSWorld (Yang et al., 2026b) simulates data science workflow steps, routes operations to simulation by their cost, and trains on about 8K transitions with RL. MLE-World predicts a whole program's terminal output from 497K logged executions and routes by its own confidence, so an uncertain program runs however cheap it is.

Test time scaling and search. Sampling (Brown et al., 2024; Snell et al., 2024), tree search (Yao et al., 2023; Zhou et al., 2023), revision from feedback (Shinn et al., 2023; Chen et al., 2024), synthesis (Khairi et al., 2026), and longer interaction (Shen et al., 2025) all assume cheap feedback; in MLE, feedback requires a training run, so MLE agents either execute every candidate (Nam et al., 2026) or rank candidates without a trace (Zheng et al., 2026). We instantiate these strategies with a world model that predicts execution feedback, so candidates are compared by predicted feedback rather than by running them, and MLE-World's confidence decides when to trust it (Srinivas et al., 2009).

3 BACKGROUND

3.1 AGENT SCAFFOLD

AIDE. We build on AIDE (Jiang et al., 2025) (although MLE-World is compatible with other agent scaffolds), the reference scaffold of MLE-bench (Chan et al., 2025) and the base of most later MLE agents (Liu et al., 2025; Toledo et al., 2026; Ou et al., 2025). AIDE searches for a program s that solves task τ over a tree whose nodes are programs and whose edges are three operators: Draft writes a new program, Debug rewrites a buggy parent, and Improve makes one atomic change to a valid parent. For fixed execution settings, the real executor E returns terminal output $o = E(\tau, s)$: training logs, the error trace on failure, and the validation score the program computes on a held out split of the training data. An LLM parser reads o and returns a bug flag and the score. The policy drafts a few initial solutions, debugs buggy nodes up to a fixed depth, and otherwise improves the best valid node; the node with the best validation score is submitted and graded on the hidden test set against the leaderboard. Every node costs one execution.

3.2 WORLD MODELS FOR LLM AGENTS

LLM world models. A language world model predicts the observation resulting from an action without executing it (Chae et al., 2025; Gu et al., 2024). In the general formulation, an LLM world model conditions on the observation–action history $(o_1, a_1, \ldots, o_t, a_t)$. Because each candidate program is self-contained, MLE-World uses only the current program and execution context to predict its terminal output, without requiring the preceding interaction history:

$$\hat{o}_{t+1} = f_\phi(\kappa, o_t, a_t).$$

Here f_ϕ is the world model with parameters ϕ, and κ provides the context needed to predict the terminal output, such as the environment state. We treat the complete candidate program as the current observation o_t and its execution as the action a_t. The model predicts the resulting terminal output as the next observation $\hat{o}_{t+1}$. Section 4 details the model's input and structured output format.

4 MLE-WORLD

MLE-World implements this prediction with a language model h_ϕ whose input x explicitly encodes the program and execution context. Before execution, it predicts a richer record y that augments terminal output o_{t+1} with three structured fields:

$$\begin{aligned} x &= \big\langle \underset{\text{program}}{o_t}, \underset{\text{packages}}{P_{\text{env}}}, \underset{\text{data overview}}{D_\tau}, \underset{\text{time limit}}{T} \big\rangle, \\ y &= \big\langle \underset{\text{exception type}}{c}, \underset{\text{terminal output}}{o_{t+1}}, \underset{\text{validation metric}}{m}, \underset{\text{submission flag}}{b} \big\rangle, \end{aligned} \tag{1}$$

Here we write $s = o_t$ for the program and $o = o_{t+1}$ for its terminal output, matching Section 3.1, and use s and o below. The fields of y are decoded in the order shown, with $c \in \mathcal{C} = \{\texttt{null}, \texttt{ValueError}, \texttt{ImportError}, \ldots\}$ (all 82 classes in Appendix C). Each exception label is one added token, decoded first. Hats denote predictions: $\hat{y} = h_\phi(x) = \langle \hat{c}, \hat{o}, \hat{m}, \hat{b} \rangle$; $\hat{o}$ is the predicted terminal output. We write x_s for the full input associated with program s, and $x_i = x_{s_i}$ for candidate i. We finetune Qwen3.5-4B with standard supervised finetuning (SFT) on logged pairs $(x, y) \in \mathcal{D}$, one execution per sequence, minimizing the negative log likelihood of the output tokens,

$$\mathcal{L}(\phi) = -\,\mathbb{E}_{(x,y)\sim\mathcal{D}}\big[\log p_\phi(y \mid x)\big] \tag{2}$$

where $p_\phi(y \mid x)$ is the conditional output distribution defined by h_ϕ (training configuration in Appendix B). At search time, these predictions drive candidate selection and the Debug and Improve operators, and the gate of Section 5.2 decides when to obtain real feedback instead.

Input. Outcomes depend on the data, available libraries, and execution budget as well as the code. MLE-World receives the program s, installed packages and versions P_{env}, AIDE's data overview D_τ (a file tree summary, CSV dimensions, and column summaries with example values), and time limit T.

Target. Predicting the terminal output alone is not enough. Without an explicit exception type, the model must infer from free text whether the run crashed, and AIDE reads the validation score only through a separate feedback model that extracts it from the output (Section 3.1). We therefore predict both as their own fields, whose labels AIDE already logs. Adding them raises agreement on terminal output from 59.4% to 62.8% and metric recall from 84.3% to 89.5% (Appendix D). The exception type also yields the outcome distribution behind our confidence score, defined next.

Confidence score from the first output token. The first decoded token is the predicted exception class $\hat{c} \in \mathcal{C}$ of Equation (1), which routes search to Debug or Improve. Its token probabilities, renormalized over $\mathcal{C}$, define the outcome distribution $p_\phi(c \mid x)$ and the scores

$$p_{\max}(x) = \max_{c \in \mathcal{C}} p_\phi(c \mid x), \qquad u(x) = 1 - p_{\max}(x), \qquad p_{\text{clean}}(x) = p_\phi(c = \texttt{null} \mid x). \quad (3)$$

$p_{\max}$ is the confidence in the most likely outcome, the uncertainty score u is the mass on all others, and p_{clean} is the probability of a clean run, equal to $p_{\max}$ when a clean run is most likely. Across quintiles of $p_{\max}$ on held out programs, every predicted field matches the real run more often as $p_{\max}$ rises (Figure 7), so $p_{\max}$ is a suitable confidence score. Search uses this outcome distribution twice: UCB (Section 5.1) uses p_{clean} to score candidates, and the gate (Section 5.2) uses $p_{\max}$ to decide whether the chosen one must run.

Data. Every executed AIDE node yields one training pair (x, y). We run AIDE with five agent models from 4B to 122B parameters on the 17 MLE-bench tasks and keep failed episodes, since a world model trained only on clean runs is least accurate on buggy candidates, where the search needs it most, and mixing agent policies makes a simulator robust to the agent that later uses it (Li et al., 2026). Appendix A reports statistics of the training data (Table 5).

5 Search guided by MLE-World

We build five world model guided TTS policies on MLE-World's predictions (Section 5.1), then add a confidence gate that works with any of them and decides when a prediction needs a real run (Section 5.2).

5.1 Test time scaling on predicted outputs

Predicted outputs let a step screen many candidates and run one. MLE-World stands in for execution when comparing candidates: a step draws N candidates, predicts their outputs $\hat{y}_i = h_\phi(x_i)$, and executes only one. Each of our five policies reads these predictions, not only the code (Figure 2; algorithms in Appendix J). *Best-of-N (BoN)* keeps the best predicted validation score (Brown et al., 2024; Nam et al., 2026). *Sequential* revises each candidate from the previous one's predicted error trace, log, and score, where prior revision uses real feedback (Snell et al., 2024; Chen et al., 2024; Nam et al., 2026). *Pairwise* extends ForeAgent (Zheng et al., 2026), which compares candidates without execution feedback: an LLM judge compares two candidates at a time with their predicted terminal outputs until one remains. *Fusion* synthesizes a new program from the pool and its predicted terminal outputs (Khairi et al., 2026; Nam et al., 2026). *UCB* adds an exploration bonus from MLE-World's confidence to the predicted score, as described next.

UCB: greedy selection can discard promising programs. BoN selects the best predicted validation score without accounting for prediction reliability. If MLE-World underestimates a promising program's quality, greedy selection can discard it before execution provides feedback for subsequent search.

Outcome uncertainty supplies an inexpensive exploration signal. GP-UCB balances estimated value with the objective's posterior standard deviation (Srinivas et al., 2009). We use outcome uncertainty $u(x)$ as an empirical exploration proxy, not a formal confidence bound. Lower confidence accompanies less accurate metric predictions on held out programs (Figure 7), so the bonus favors potentially underestimated candidates at no additional inference cost.

The bonus balances predicted quality and exploration. At search step t the agent draws N candidates; $\mathcal{S}_t$ holds those predicted to complete normally, write a submission, and have a finite metric accepted by the reviewer. We write $\hat{m}(s_i) = \hat{m}_i$ for the metric field of $\hat{y}_i$. Greedy selection on this pool chooses $\arg\max_{s \in \mathcal{S}_t} d_\tau \hat{m}(s)$ ($d_\tau = +1$ for a maximized metric and -1 for a minimized metric). An uncertain prediction may underestimate the true quality, so adding the bonus gives such a candidate a chance to beat a confidently predicted one with a slightly better estimate:

$$s_t^\star = \arg\max_{s \in \mathcal{S}_t} \big[v(s) + \beta u(x_s) \big], \qquad v(s) = \frac{d_\tau \big(\hat{m}(s) - m_{\text{ref}} \big)}{\alpha_t}, \tag{4}$$

where $s_t^\star$ is selected and $v(s)$ measures predicted gain over m_{ref} in units of $\alpha_t > 0$. Both terms are set at step t and shared within its pool; with a verified reference, $\alpha_t = \max(|m_{\text{ref}}|, 0.01)$ (fallbacks in Appendix J). The UCB runs decode greedily, making `null` the most probable class on $\mathcal{S}_t$; therefore $u(x_s) = 1 - p_{\max}(x_s) = 1 - p_{\text{clean}}(x_s)$ there. The weight $\beta \geq 0$ controls exploration, and the gate (Section 5.2) runs low confidence selections for real. Without an eligible metric, UCB ranks by $p_{\text{clean}}(x_s)$, whereas BoN returns the first usable program or asks an LLM judge (Appendix J).

5.2 Execution gating by uncertainty

Real runs are spent where predictions are least reliable. Predictions are cheap but not always right: on held out programs, 22% of predicted terminal outputs and 14% of predicted exception classes disagree with the real run (Table 1), and such errors can mislead search (Janner et al., 2019). Because accuracy rises with confidence (Section 4), after any of the five policies selects a candidate, the gate requests real feedback when

$$p_{\max}(x_{s_t^\star}) < \gamma, \tag{5}$$

or when confidence is missing, where $s_t^\star$ is the program returned by the search policy at step t and $\gamma \in (0, 1)$ is a confidence threshold. Otherwise the predicted output guides the next Debug or Improve step. We always execute a candidate with a new best predicted validation metric, regardless of confidence, because the best verified candidate supplies the final submission.

Thresholds follow the cost of each predicted outcome. We set $\gamma = \gamma_{\text{s}}$ for predicted successes and $\gamma = \gamma_{\text{f}}$ for failures, because their runs cost very different amounts of time. A failing program usually crashes within seconds, so we can afford to run many of them, and each run returns the exact error trace to repair. A successful program trains to completion, which is where execution time goes, so a lower γ_{s} skips more of these long runs. Programs predicted to time out are never run, since a timeout is the most costly outcome.

6 Experiments

The experiments answer three questions: **(Q1)** How accurately does MLE-World predict execution outcomes, and at what inference cost? (Section 6.1) **(Q2)** Does world model guided search improve submitted solutions, and do predicted outputs and the exploration bonus each help? (Section 6.2) **(Q3)** Does confidence identify unreliable predictions, and what tradeoff between quality and execution cost does gating achieve? (Section 6.3) Our main experiments use the same 17 of the 22 MLE-bench Lite tasks (Chan et al., 2025): following Yang et al. (2026a), we exclude five tasks on which our agents rarely make progress (task selection and statistics of the training data in Appendix A). We run agent experiments (Sections 6.2 and 6.3) with three seeds per task and, following MLE-bench (Chan et al., 2025), select the submission with the best real validation score in each trial for grading on the hidden test set. We report the mean over seeds with its standard error (SE).

6.1 MLE-World predicts execution accurately at low cost

We answer **(Q1)** by evaluating MLE-World in isolation on a disjoint held out set: per field accuracy of the predicted output and inference cost, against much larger prompted models.

Table 1: **MLE-World leads all prediction fields.** Accuracy (%) on 1,700 held out programs; ± SE from 2,000 program bootstrap resamples. Exception class: exact match; terminal output and crash class: judged match; metric: within 5% relative error. Corpus baselines: Appendix F.

	Terminal output	Exception class	Crash class	Validation metric
Prompted models				
DeepSeek	41.1±1.1	57.2±1.2	48.2±1.5	18.6±1.7
Coder	17.8±0.9	27.5±1.1	13.2±1.0	16.3±1.7
Corpus, no model				
Task majority	–	34.2±1.2	–	55.2±2.3
Nearest program	28.0±1.1	41.1±1.2	14.0±1.0	55.6±2.2
Parent record	22.3±1.0	34.2±1.2	6.7±0.7	48.8±2.3
MLE-World	**77.9**±1.0	**86.4**±0.8	**76.4**±1.3	**78.1**±1.9

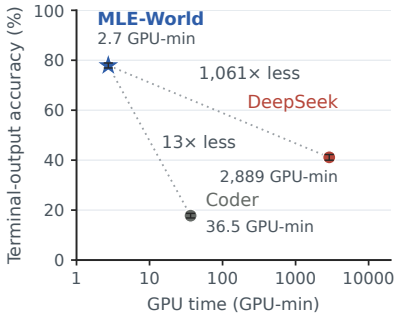


Figure 3: **Accurate prediction at low inference cost.** Terminal output accuracy versus GPU time (wall clock × GPUs; log scale). Bars: ± SE. Ratios are based on generation time for all models.

Training setup. We finetune Qwen3.5-4B on the 497,056 training programs of Section 4, with its input and target of four fields (hyperparameters and hardware in Appendix B). Adding the exception class and validation metric to the target improves terminal output agreement, so we use this four field target (ablation in Appendix D).

Evaluation setup. We evaluate on 1,700 held out programs, 100 per task and balanced across agent stages. No held out program has an exact or near duplicate copy in training (Appendix E describes the deduplication), but the tasks are shared by design. As in model based RL, where a world model predicts new actions in the environment it was trained on (Ha & Schmidhuber, 2018; Hafner et al., 2019; Janner et al., 2019), the task is the environment and each new program is an action. We compare against two strong prompted coder models, DeepSeek-V4-Flash-0731 (DeepSeek) and Qwen3-Coder-Next (Coder), and three corpus baselines that copy records from MLE-World's training data without a model: task majority, nearest program, and parent record (see Appendix F for details of each method). Because exact matching misses rewordings and conflates crashes with different causes, a Coder judge scores terminal output and crash class; it agrees with held out human labels on 96.7% (Appendix I).

1) MLE-World predicts outcomes beyond task priors and record retrieval. Table 1 shows that MLE-World's predicted terminal output matches the observed output on 77.9% of held out programs, compared with 41.1% for DeepSeek, the strongest prompted baseline. The corpus controls clarify what the model learns: task level statistics already predict many validation scores, but retrieving a similar program's record rarely reproduces its failure cause, since a one line difference in code can produce a different error. MLE-World's advantage on both fields supports learning relationships between programs and their execution outcomes beyond those captured by the tested baselines. As shown in Figure 3, its GPU inference time is 13–1,061× lower than that of the prompted models, making these predictions practical for screening multiple candidates.

2) MLE-World misses what reading the program cannot reveal. Table 12 shows that MLE-World leads both prompted models at every stage and outcome. Comparing it with Table 5, its errors follow what the input can show, not how often an outcome appears in training. A `SyntaxError` is visible in the code, so MLE-World predicts every one, although they are only 8.2% of training crashes. Timeouts are its weakest outcome because run time must be estimated from model size, epochs, and data size.

6.2 Observed solution quality with world model guided search

We answer **(Q2)** in three steps: full search runs without confidence gating, single decisions on fixed task states, and UCB's exploration bonus in isolation.

Policy	Mean percentile ↑	Δ vs. Base	Valid (%) ↑
Base AIDE	36.9±3.3	–	78.4±2.0
MLE-World guided (8 h)			
Sequential	41.1±4.0	+4.2±3.3	90.2±5.2
Pairwise	42.8±3.3	+5.9±4.5	92.2±2.0
Fusion	**47.2**±2.3	**+10.4**±3.7	**96.1**±2.0
BoN	39.5±3.1	+2.6±3.7	88.2±0.0
UCB	45.1±1.1	+8.3±4.5	88.2±0.0

Table 2: **Search without confidence gating.** 17 tasks × three seeds; 8 h. Invalid submissions score zero. Mean ± SE across seed means; Δ: gain over Base AIDE. Bold: best 8 h mean.

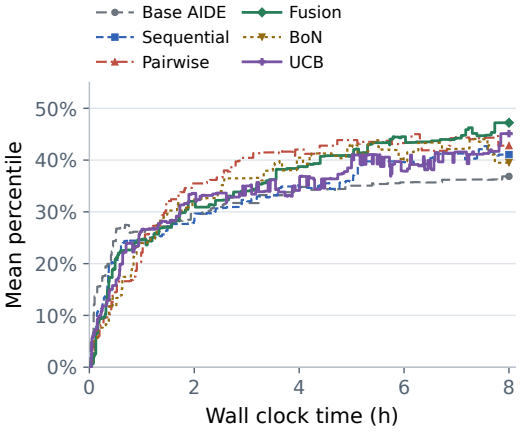


Figure 4: **Search quality over time.** Percentile of the validation-selected submission, carried forward and zero until valid.

Setup. Each policy has 51 eight-hour trials (17 tasks × three seeds), with Qwen3.5-9B writing code in AIDE. The five guided policies (Figure 2) sample $N = 4$ candidates per step and return one program, selected from the pool or synthesized by Fusion; without the gate, it is always executed. Base AIDE writes and runs one program per step. We select $\beta = 0.5$ for UCB through tuning (Appendix L). We report mean leaderboard percentile because medal thresholds amplify noise and hide progress on hard tasks (Hambardzumyan et al., 2026; Appendix Q).

1) All five policies have higher observed means than the agent without a world model. Fusion reaches 47.2 mean percentile versus 36.9 for Base AIDE[1]; all guided policies have higher observed means (Table 2). They also spend less time on crashes and timeouts (Figure 6), consistent with predictions steering execution toward programs that run and score.

2) Predicted terminal output helps search policies. To isolate the benefit of predicted terminal output, we compare each policy with and without it from the same starting states. On 150 shared states, Pairwise and Fusion receive the same four input candidates; Sequential makes three revisions from the first. Fusion adds one synthesis call before retries; Pairwise uses three judge calls. With predicted terminal output, Fusion has a higher observed mean than with code alone: 20.6 versus 14.0 (Table 3). This suggests that predicted terminal output improves search by helping policies select or synthesize better candidates within each step.

Policy	w/o predicted terminal output	w/ predicted terminal output
Pairwise	13.0	15.4
Sequential	10.4	15.3
Fusion	14.0	**20.6**

Table 3: **Value of predicted terminal output.** Mean percentile on 150 shared states, 17 tasks; invalid outputs score zero. Sequential always selects by predicted metric. Details: Table 20.

6.3 Confidence gating: execution savings and observed quality

We answer **(Q3)** by running each selection policy with and without the confidence gate.

Gating setup. For each of the five policies, we apply the gate of Section 5.2 using thresholds selected on a separate 3 h development run (Appendix M). All other settings match the ungated runs.

1) Prediction fidelity rises steadily with confidence. We group the held out programs from Section 6.1 into quintiles of $p_{\max}$. Figure 7 shows that match rates for validation metrics, exception classes, and terminal outputs rise with confidence, so confidence can prioritize verification.

[1]Real-execution BoN runs all four candidates and reaches 41.9 at 8 h (Appendix O).

3) The exploration bonus yields a higher observed mean. UCB and BoN also differ in eligibility and fallback. We isolate the bonus by varying only Improve-stage β: 0 (greedy) versus 1 (UCB), with other settings fixed (Appendix K.1). With the bonus, the observed mean percentile is 45.4, versus 38.6 for greedy selection (Figure 5). This suggests that the bonus can help search by favoring candidates whose predictions are uncertain.

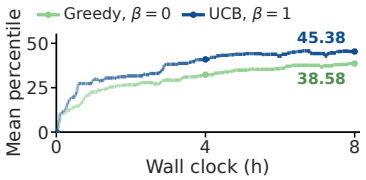


Figure 5: **The exploration bonus yields a higher observed mean.** All other settings are fixed; only Improve-stage β varies (Figure 8a).

Mean percentile ↑

Policy	Ungated	Gated	Δ
Base AIDE	36.9	–	–
Sequential	41.1	47.7	+6.6±4.7
Pairwise	42.8	48.5	+5.7±2.4
Fusion	47.2	46.4	-0.8±7.7
BoN	39.5	47.2	+7.7±3.2
UCB	45.1	49.2	+4.1±1.5

Real execution (h/trial) ↓

Policy	Ungated	Gated	Δ	Saved (%)
Base AIDE	6.65	–	–	–
Sequential	5.23	3.62	-1.61±0.29	30.8
Pairwise	5.41	2.91	-2.51±0.31	46.3
Fusion	5.20	3.20	-2.00±0.11	38.5
BoN	6.10	4.49	-1.62±0.15	26.5
UCB	5.87	4.45	-1.42±0.06	24.2

Table 4: **Confidence gating reduces real execution.** 17 tasks × three seeds, 8 h; invalid submissions score zero. Δ: gated minus ungated, ± SE across the three paired seed means (17 tasks per seed). Wall clock curves: Figure 12 (Appendix O).

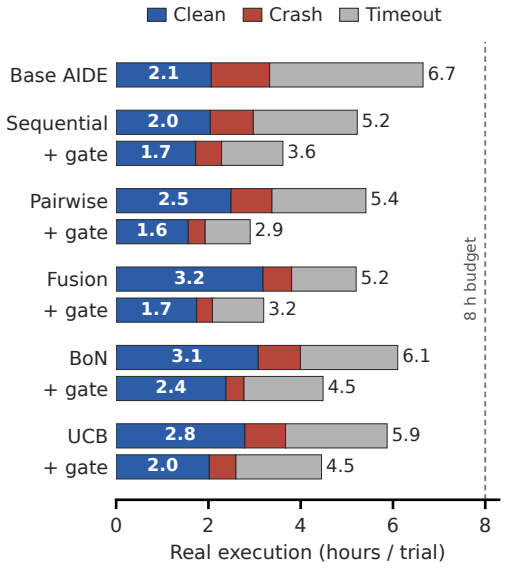


Figure 6: **Execution savings by outcome.** Mean hours per trial, with seeds averaged within equally weighted tasks. Inference and search overhead excluded.



Figure 7: **Prediction fidelity rises with confidence.** Accuracy by $p_{\max}$ quintile on 1,700 held out programs: (a) metric within 5% relative error (484 references; missing predictions are misses), (b) exact exception class, (c) judged terminal output. Bars: ± task bootstrap SE (2,000 resamples, fixed bins).

2) Confidence gating reduces real execution. Table 4 shows that across the five policies, gating saves 24–46% of real execution time, with higher observed mean percentiles for four of them (trajectories in Figures 12 and 11). This suggests that confident predictions are usually right, so

skipping their runs costs little, and the saved time buys more search steps within the same budget (Appendix O).

3) Confidence contributes beyond random skipping. As shown in Table 17 (Appendix N), for BoN, confidence gating uses 19% less real execution than a random gate that skips runs with a fixed probability, at a similar final percentile. This suggests that the savings come from which runs the gate skips, not only how many.

7 LIMITATIONS AND CONCLUSION

Limitations. Our main evaluation is in distribution: new programs on 17 tasks seen in finetuning (Appendix E). On three tasks excluded from finetuning, the 4B MLE-World, without reasoning, still predicts terminal output more accurately than much larger prompted models (53.3% versus 42.1% for DeepSeek with reasoning and 21.1% for Coder; Appendix R). This opens a generalizable world model as future work, for example by adding reasoning or scaling up the model. Training uses about 500,000 prior executions, including earlier trajectories from a 9B agent but no reported runs, favoring settings with accumulated logs. Zero shot baselines and Base AIDE lack this task knowledge; nonparametric corpus baselines trail MLE-World (Table 1), while Base AIDE with corpus access remains untested. Real execution determines final scores; prediction errors can misdirect search. Timeouts are the least accurately predicted outcome, and a missed timeout still costs a full run; improving runtime prediction is future work.

Conclusion. A 4B model finetuned on logged executions predicts the outcomes of new programs more accurately than much larger prompted models, at 13–1,061$\times$ less GPU inference time. This makes test time scaling affordable for MLE agents: all five world model guided policies have higher observed mean percentiles than the agent without a world model. Gating uses 24–46% less real execution, with higher observed means for four of the five policies.

ETHICS STATEMENT

This work raises no ethical concerns. Its experiments use only public Kaggle competition data distributed with MLE-bench and involve no human subjects or personal data.

REPRODUCIBILITY STATEMENT

Appendices A to G give the training data, training configuration, deduplication and holdout, decoding and the prediction prompt, and output parsing; Appendices I, J, M, and N give the judge prompt and validation, search algorithms, gate thresholds, and random control. Agent results state seeds, budgets, and evaluation protocols. MLE-World was trained on 8 NVIDIA H200 GPUs (full training configuration in Appendix B). We will release the MLE-World checkpoint, the execution corpus with its held out split, all prompts, and the agent and evaluation code.

LLM USAGE

LLMs are part of the method: Qwen3.5 models generate agent programs and Qwen3-Coder-Next judges outputs. An LLM coding agent (GPT-5.6 Sol) developed the judge prompt under the protocol of Appendix I. The authors also used LLM assistants to write experiment and analysis code and to edit the text, and remain responsible for all content.

REFERENCES

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

Hyungjoo Chae, Namyoung Kim, Kai Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *International Conference on Learning Representations*, volume 2025, pp. 63707–63738, 2025.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, volume 2025, pp. 50466–50494, 2025.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations*, volume 2024, pp. 8746–8825, 2024.

Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. WebEvolver: Enhancing web agent self-improvement with co-evolving world model. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 8959–8975, 2025.

Thomas Simon Foster, Bassel Al Omari, Tingchen Fu, Thomas Mann, Carl Domond, Lucia Cipolina-Kun, Bhavul Gauri, Muna Aghamelu, Alexander D. Goldie, Eryk Helenowski, et al. AI research preference models. *arXiv preprint arXiv:2608.13940*, 2026.

Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, et al. Is your LLM secretly a world model of the internet? model-based planning for web agents. *arXiv preprint arXiv:2411.06559*, 2024.

Yiming Guan, Rui Yu, John Zhang, Lu Wang, Chaoyun Zhang, Liqun Li, Bo Qiao, Si Qin, He Huang, Fangkai Yang, et al. Computer-using world model. *arXiv preprint arXiv:2602.17365*, 2026.

David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.

Karen Hambardzumyan, Nicolas Baldwin, Edan Toledo, Rishi Hazra, Michael Kuchnik, Bassel Al Omari, Thomas Simon Foster, Anton Protopopov, Jean-Christophe Gagnon-Audet, Ishita Mediratta, et al. AIRA_2: Overcoming bottlenecks in AI research agents. *arXiv preprint arXiv:2603.26499*, 2026.

Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. *Advances in Neural Information Processing Systems*, 32, 2019.

Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.

Ammar Khairi, Daniel Dsouza, Marzieh Fadaee, and Julia Kreutzer. Making, not taking, the best of N. In *International Conference on Learning Representations*, volume 2026, pp. 149049–149083, 2026.

Stepan Kulibaba, Artem Dzhalilov, Roman Pakhomov, Oleg Svidchenko, Alexander Gasnikov, and Aleksei Shpilman. KompeteAI: Accelerated autonomous multi-agent system for end-to-end pipeline generation for machine learning problems. *arXiv preprint arXiv:2508.10177*, 2025.

Yixia Li, Hongru Wang, Jiahao Qiu, Zhenfei Yin, Dongdong Zhang, Cheng Qian, Zeping Li, Xiaoteng Ma, Guanhua Chen, and Heng Ji. From word to world: Can large language models be implicit text-based world models? In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8084–8111, 2026.

Zexi Liu, Yuzhu Cai, Xinyu Zhu, Yujie Zheng, Runkun Chen, Ying Wen, Yanfeng Wang, Weinan E, and Siheng Chen. ML-Master: Towards AI-for-AI via integration of exploration and reasoning. *arXiv preprint arXiv:2506.16499*, 2025.

Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan Arik, and Tomas Pfister. MLE-STAR: Machine learning engineering agent via search and targeted refinement. *Advances in Neural Information Processing Systems*, 38:116692–116712, 2026.

Yixin Ou, Yujie Luo, Jingsheng Zheng, Lanning Wei, Zhuoyun Yu, Shuofei Qiao, Jintian Zhang, Da Zheng, Yuren Mao, Yunjun Gao, et al. AutoMind: Adaptive knowledgeable agent for automated data science. *arXiv preprint arXiv:2506.10974*, 2025.

Junhong Shen, Hao Bai, Lunjun Zhang, Yifei Zhou, Amrith Setlur, Shengbang Tong, Diego Caples, Nan Jiang, Tong Zhang, Ameet Talwalkar, et al. Thinking vs. doing: Agents that reason by scaling test-time interaction. *arXiv preprint arXiv:2506.07976*, 2025.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Niranjan Srinivas, Andreas Krause, Sham M. Kakade, and Matthias Seeger. Gaussian process optimization in the bandit setting: No regret and experimental design. *arXiv preprint arXiv:0912.3995*, 2009.

Shuang Sun, Huatong Song, Lisheng Huang, Jinhao Jiang, Ran Le, Zhihao Lv, Zongchao Chen, Yiwen Hu, Wenyang Luo, Wayne Xin Zhao, et al. SWE-World: Building software engineering agents in Docker-free environments. *arXiv preprint arXiv:2602.03419*, 2026.

Aditya Thimmaiah, Jiyang Zhang, Jayanth Srinivasa, Junyi Jessy Li, and Milos Gligoric. LLMs lean on priors, not programming language semantics. *arXiv preprint arXiv:2510.03415*, 2025.

Edan Toledo, Karen Hambardzumyan, Martin Josifoski, Rishi Hazra, Nicolas Baldwin, Alexis Audran-Reiss, Michael Kuchnik, Despoina Magka, Minqi Jiang, Alisia Lupidi, et al. AI research agents for machine learning: Search, exploration, and generalization in MLE-bench. *Advances in Neural Information Processing Systems*, 38:35309–35348, 2026.

Ruoyao Wang, Graham Todd, Ziang Xiao, Xingdi Yuan, Marc-Alexandre Côté, Peter Clark, and Peter Jansen. Can language models serve as text-based world simulators? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1–17, 2024.

Colin White, Arber Zela, Robin Ru, Yang Liu, and Frank Hutter. How powerful are performance predictors in neural architecture search? *Advances in Neural Information Processing Systems*, 34:28454–28469, 2021.

Sherry Yang. Position: World models as an intermediary between agents and the real world. In *Forty-third International Conference on Machine Learning Position Paper Track*, 2026.

Sherry Yang, Joy He-Yueya, and Percy Liang. Reinforcement learning for machine learning engineering agents. In *International Conference on Learning Representations*, volume 2026, pp. 156989–157011, 2026a.

Zherui Yang, Fan Liu, and Hao Liu. DSWorld: A data science world model for efficient autonomous agents. *arXiv preprint arXiv:2607.15901*, 2026b.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36:11809–11822, 2023.

Xiao Yu, Baolin Peng, Ruize Xu, Michel Galley, Hao Cheng, Suman Nath, Jianfeng Gao, and Zhou Yu. Dyna-Think: Synergizing reasoning, acting, and world model simulation in AI agents. *arXiv preprint arXiv:2506.00320*, 2025.

Jingsheng Zheng, Jintian Zhang, Yujie Luo, Yuren Mao, Yunjun Gao, Lun Du, Huajun Chen, and Ningyu Zhang. Can we predict before executing machine learning agents? In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3941–3974, 2026.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyan Luo. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 400–410, 2024.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.

Yuxin Zuo, Zikai Xiao, Li Sheng, Fei Huang, Jianhong Tu, Yuxuan Liu, Tianyi Tang, Xiaomeng Hu, Yang Su, Qingfeng Lan, et al. Qwen-AgentWorld: Language world models for general agents. *arXiv preprint arXiv:2606.24597*, 2026.

A TRAINING DATA

Table 5 breaks the training set down by generator model, agent stage, outcome, and task. Every task in the holdout is also a training task; the held out programs themselves are excluded (Appendix E).

Task selection. MLE-bench Lite contains 22 tasks. Following Yang et al. (2026a), who evaluate on the tasks where their base model could generate a valid initial solution, we keep the 17 tasks on which our agents produce working programs often enough to train on and to search. On the four excluded image tasks (`aptos2019-blindness-detection`, `histopathologic-cancer-detection`, `ranzcr-clip-catheter-line-classification`, and `siim-isic-melanoma-classification`), only 2.3–11.6% of deduplicated executions across all

agent models finish cleanly with a submission, against 38.6% on the 17 kept tasks (at least 13.3% on each); our corpus contains no executions of the fifth, `the-icml-2013-whale-challenge-right-whale-redux`.

Table 5: **Training data statistics.** Most training programs fail: 52.3% crash and 7.0% time out, so MLE-World sees failures more often than clean runs. Counts over the 497,056 training programs, each recovered to its task, generator model, and stage from the deduplicated node pool. Outcome, exception class, and labels come from the training target; token counts use the Qwen3.5-4B tokenizer, and the prompt includes the system message.

	Programs	%
All training programs	497,056	100.0
Generator model		
Qwen3-Coder-Next	209,573	42.2
Qwen3.5-4B	129,907	26.1
Qwen3.5-122B-A10B	77,239	15.5
Qwen3.5-9B	54,815	11.0
Qwen3.6-35B-A3B	25,522	5.1
Agent stage		
Draft	106,542	21.4
Improve	183,875	37.0
Debug	206,639	41.6
Outcome		
Clean run	202,408	40.7
Crash	259,884	52.3
Timeout / interrupt	34,764	7.0
Crash exception class (% of crashes)		
ValueError	72,882	28.0
TypeError	36,718	14.1
NameError	25,578	9.8
KeyError	21,987	8.5
SyntaxError	21,380	8.2
AttributeError	20,846	8.0
RuntimeError	15,362	5.9
FileNotFoundError	10,352	4.0
Other	34,779	13.4
Labels		
Numeric validation metric	193,562	38.9
Submission written	199,081	40.1
Task	Programs	
aerial-cactus-identification	22,705	
denoising-dirty-documents	26,357	
detecting-insults-in-social-commentary	31,255	
dog-breed-identification	30,278	
dogs-vs-cats-redux-kernels-edition	53,010	
jigsaw-toxic-comment-classification-challenge	15,724	
leaf-classification	16,603	
mlsp-2013-birds	57,706	
new-york-city-taxi-fare-prediction	15,589	
nomad2018-predict-transparent-conductors	37,299	
plant-pathology-2020-fgvc7	27,075	
random-acts-of-pizza	45,494	
spooky-author-identification	21,896	
tabular-playground-series-dec-2021	28,218	
tabular-playground-series-may-2022	20,160	

Table 5: Training data statistics (continued).

text-normalization-challenge-english-language	26,783
text-normalization-challenge-russian-language	20,904
Length, median (90th pct.)	
Prompt tokens	3,421 (5,406)
Target tokens	186 (617)
Program lines	203 (369)

B TRAINING CONFIGURATION

We train all parameters for five epochs with LlamaFactory (Zheng et al., 2024) on 8 H200 GPUs (43 hours) and use the final checkpoint. The input is the program with its importable packages and versions, the task's data overview, and the execution time limit, which lets the model predict timeouts. Table 6 lists the LlamaFactory configuration of the MLE-World checkpoint evaluated in Section 6. Each exception class is a single added token (`EXC_ValueError`, ...), so the class is decoded as one token rather than spelled out. Holdout loss was logged during training but not used to select the checkpoint.

Table 6: **Training configuration.** SFT of all parameters of Qwen3.5-4B; we evaluate the final checkpoint (step 6,685).

Base model	Qwen3.5-4B
Method	SFT of all parameters (LlamaFactory), vision tower frozen
Training data	497,056 programs (Table 5)
Chat template	`qwen3_5`, thinking disabled
Added tokens	82 exception class tokens (`EXC_None`, `EXC_ValueError`, ...; Appendix C)
Sequence length	12,288 tokens, packed without attention across examples
Epochs / steps	5 / 6,685
Batch size	128 packed sequences (8 GPUs $\times$ 1 $\times$ 16 accumulation)
Optimizer	AdamW (fused), weight decay 0.01, gradient clipping 1.0
Learning rate	1×10^{-5}, cosine schedule, 5% warmup
Precision	bf16 mixed precision, FP32 weights and optimizer states
Memory	Gradient checkpointing, FlashAttention-2, Liger kernels
Hardware	8 NVIDIA H200 GPUs (4 nodes $\times$ 2), 42.7 hours
Final loss	0.079 train, 0.081 holdout

C EXCEPTION CLASSES

Table 7 lists the set $\mathcal{C}$ of Equation (1): the 82 exception classes MLE-World can predict, each added to the tokenizer as one token (`EXC_`*name*). The set covers every exception class that occurs in the training corpus, named as the logged traceback reports it; `None` is the no exception class, written `null` in the main text. Because each class is a single token, the first decoded token gives a distribution over all 82 classes, from which Equation (3) reads $p_{\max}$ and p_{clean}.

D TARGET ABLATION

MLE-World predicts an exception class and a validation metric alongside the terminal log (Section 4). This ablation asks whether these fields add information or merely duplicate what the log already contains.

Setup. We fix the backbone, training corpus, split, and decoding, and vary only the target. Starting from terminal output plus a submission flag, we add the exception class and then the validation metric. We also compare untrimmed logs with the default window of the first and last 2,500 characters, and we train a target of metric plus submission flag with no terminal output. To keep each

Table 7: **Exception classes.** The 82 classes of $\mathcal{C}$, in token order; each is one added token EXC_*name*.

None (null)	MissingCorpusError
AcceleratorError	ModuleNotFoundError
ArrowInvalid	NameError
ArrowTypeError	NotADirectoryError
AssertionError	NotFittedError
AttributeError	NotImplementedError
AxisError	OSError
BadZipFile	OptionError
BdbQuit	OutOfBoundsDatetime
BracketError	OutOfMemoryError
CalledProcessError	OverflowError
CatBoostError	ParserError
CompressionError	PicklingError
DTypePromotionError	ReadError
DateParseError	RecursionError
DeferredCudaCallError	RepositoryNotFoundError
EOFError	RuntimeError
EmptyDataError	SameFileError
Error	SpecificationError
Exception	StopIteration
FileExistsError	SyntaxError
FileNotFoundError	SystemExit
FloatingPointError	TabError
HTTPError	TerminatedWorkerError
ImportError	TimeoutError
IndentationError	TorchRuntimeError
IndexError	TqdmKeyError
IndexingError	TypeError
IntCastingNaNError	UFuncTypeError
InvalidIndexError	URLError
InvalidParameterError	UnboundLocalError
InvalidURL	UnicodeDecodeError
InvalidVersion	UnicodeEncodeError
IsADirectoryError	UnidentifiedImageError
JSONDecodeError	UnpicklingError
KeyError	UnsupportedOperation
LightGBMError	ValueError
LinAlgError	VersionError
LookupError	XGBoostError
MemoryError	ZeroDivisionError
MergeError	error

run affordable, the variants train on a subset of 49,791 programs drawn from nine of the tasks and are scored against executed ground truth on 450 held out programs from those nine tasks (Table 8). Absolute scores are therefore not comparable with Table 1.

Little observed difference from trimming. We observed little difference between the 2,500 character window and untrimmed logs (terminal output 59.4% vs. 59.5%, exception class 71.7% vs. 71.4%; Table 8). With one training run per condition, these point estimates do not establish equivalence.

Structured fields improve the log itself. Adding the exception class and then the validation metric raises agreement on terminal output from 59.4% to 62.8%, exception class agreement from 71.7% to 75.3%, and crash class agreement from 64.8% to 69.1%. Predicting explicit outcome fields also makes the generated log more accurate, rather than serving only as a separate output.

Table 8: **Target ablation.** Structured fields improve the predicted log, and dropping the log costs metric recall. 450 held out programs from its nine training tasks; backbone, data, and decoding are fixed. "+" adds fields cumulatively; trimmed logs retain the first and last 2,500 characters. All scores are percentages. Terminal output measures semantic agreement; exception/crash measure exact class agreement inferred from logs, with "no exception" included and crash restricted to jointly crashing cases. Metric precision uses 10% relative tolerance on commonly answered cases; recall is the fraction of cases with a numeric reference receiving a metric. Dashes indicate undefined scores from logs. Each row is one training run; at 450 programs, a rate near 60–75% has a bootstrap SE over programs of about 2 points (more in the crash and metric columns, which use subsets).

target	Terminal output	Exception class	Crash class	Validation metric (precision / recall)
terminal output (untrimmed) + submission flag	59.5	71.4	64.4	87.3 / 83.5
terminal output (trimmed) + submission flag	59.4	71.7	64.8	88.7 / 84.3
+ exception class	60.7	73.0	66.7	87.3 / 87.6
+ validation metric	**62.8**	**75.3**	**69.1**	89.0 / **89.5**
validation metric + submission flag	–	–	–	**90.0** / 77.0

Dropping the log loses coverage and feedback. A target of the metric alone reaches the highest metric precision (90.0%) but predicts a metric for only 77.0% of cases with a numeric reference, compared with 89.5% for the full target. It also provides no error trace for debugging. The full target gives up 1.0 point of precision for 12.5 points of recall and the feedback from logs the agent needs to repair failures.

E Deduplication and holdout construction

Deduplication. Agents rarely resubmit byte identical code; repeated attempts usually differ only in comments or formatting. We therefore key each program on a hash of its Python token stream with comments and blank lines removed and indentation and line breaks normalized. Programs that differ only in comments, spacing, or blank lines share a key; any change to identifiers, literals, or structure does not. Programs that fail to tokenize fall back to a hash of their raw bytes. Before deduplication, we drop executions whose outcome reflects the harness rather than the program, such as GPU contention, host memory limits, or network failures. Deduplication then collapses 757,186 executions to 602,741. When the same program produced different outcomes across runs (for example, a timeout in one run and a clean finish in another), we keep one representative per outcome rather than an arbitrary winner.

Filtering. From the 602,741 deduplicated executions we remove 43,836 on tasks outside our 17, 33,591 whose output contains a progress bar (agents are told not to print them), 13,121 whose validation metric cannot be resolved unambiguously, 11,549 cut short by the agent's overall deadline and logged as timeouts, and 1,888 with harness failures, invalid stage labels, or mislabeled interrupts. This leaves 498,756 executions.

Holdout. From these 498,756 executions, we sample 100 programs per task, balanced across agent stages (draft, improve, debug) and between fast ($\leq$5 s) and slow executions. We draw only programs without a detected exact or near duplicate among the other programs of their task (procedure below), then remove the sampled programs from training. The remaining 497,056 executions form the training set. Because of this balancing, the holdout does not follow the stage mix of training or search (Table 5).

Near duplicates. Exact keys miss programs that differ only in identifier names or in a few edits. We therefore replace identifiers with placeholders and treat two programs as duplicates when their identifier normalized abstract syntax trees hash identically, and as near duplicates when MinHash with locality sensitive hashing estimates a Jaccard similarity of at least 0.9 between their identifier

normalized token shingles (128 permutations in 16 bands of 8 rows, within each task). No exact or near duplicates were detected between holdout and training under this procedure. The probabilistic MinHash/LSH search can miss similar pairs; it does not rule out semantic similarity or overlap across tasks.

Shared tasks. Held out programs are new, but their tasks also appear in training. This mirrors model based RL, where a world model is trained and evaluated in the same environment and must predict the outcome of new actions, not new environments (Ha & Schmidhuber, 2018; Hafner et al., 2019; Janner et al., 2019). Here the task is the environment and executing a program is an action: an agent writes hundreds of new programs for the task it is solving, so MLE-World must predict unseen programs within known tasks.

F INFERENCE AND JUDGE SETTINGS

Table 9 lists the decoding settings used to predict the holdout. MLE-World and Coder decode greedily; DeepSeek samples one output per program at temperature 1.0 and top-p 0.95. MLE-World runs without reasoning. Coder is a nonreasoning model by default; we include it as a strong coding model, since predicting execution requires reading code. DeepSeek runs with reasoning enabled at maximum effort, and only its final answer, not its reasoning trace, is scored.

Table 9: **Decoding settings.** Settings used to predict the 1,700 held out programs, and for the judge that scores terminal output and crash class.

	MLE-World	Coder	DeepSeek	Judge (Coder)
Reasoning	off	off	on (max effort)	off
Temperature	0 (greedy)	0 (greedy)	1.0	1.0
Top-p	1.0	1.0	0.95	0.95
Top-k	–	–	–	40
Samples per program	1	1	1	3
Max output tokens	8,192	8,192	32,768	3,072

The judge is Coder and sees only the predicted and observed terminal outputs, not the exception labels. Coder is also a baseline. It is the least accurate model on the judged fields of Table 1, so a judge bias toward its own outputs would not favor MLE-World, but we do not rule such a bias out.

Prediction prompt. All three models receive the same system and user messages, reproduced below. The prompted baselines are zero shot, with no examples from the corpus. MLE-World emits the exception class as one added token; the prompted models write the same `EXC_` label as text.

```
--- system message ---
Predict the outcome and terminal output (stdout+stderr) of a
   Python ML script.

Input, in order:
- <code>, the script, run as runfile.py.
- <environment>, the importable packages. Authoritative: a
   library not listed is not importable, and the listed version
   is the one whose API applies.
- <data_overview>, truth for files/columns under ./input/.
- <runtime_budget>, the wall-clock limit. The script is killed
   when it is exceeded.

Output, in order and nothing else:
- <exc_type>...</exc_type>, one EXC_<ClassName> token: EXC_None
   if it runs clean, otherwise its exception class prefixed with
   EXC_ (e.g. EXC_ValueError); EXC_TimeoutError if its
   wall-clock time would exceed <runtime_budget>.
```

```
- <term_out>...</term_out>, the script's stdout+stderr. Write it
    before the two fields below -- they are read off it.
- <val_metric>...</val_metric>, the validation score reported in
    the term_out, or None if it reports none, the validation is
    invalid (e.g. scoring training data or computing the wrong
    metric), or the script raises.
- <submission_exists>true|false</submission_exists>, whether
    ./submission/submission.csv exists after the run.

Missing files or columns → predict the right Python exception,
    don't invent data.
A script can finish without raising and still write no submission
    -- decide <submission_exists> from what the code actually
    writes to ./submission/, not from whether it crashed.

--- user message ---
<code>
{program, run as runfile.py}
</code>

<environment>
{Python version; importable packages with versions; packages not
    installed or failing to import}
</environment>

<data_overview>
{file tree under ./input/ with sizes, plus columns and sample
    rows, as AIDE shows the agent}
</data_overview>

<runtime_budget>{execution time limit, e.g.
    3400s}</runtime_budget>
```

Corpus baselines. These baselines use no model. They see the same 497,056 training programs as MLE-World, or the parent's observed record, and copy records verbatim in MLE-World's target format, so the parser, judge, and bootstrap of Table 1 apply unchanged (Table 10). *Task majority* predicts, for each task and stage, the most common exception class and submission flag of the training programs and the median of their validation metrics; it writes no terminal output. *Nearest program* copies the record of the training program of the same task whose code is most similar (TF-IDF cosine over code tokens and token bigrams; median cosine 0.70). *Parent record* copies the observed record of the program's parent, which an agent already holds when it writes an Improve or Debug child; the 561 drafts and 14 programs whose parent is missing from the logs fall back to the nearest program. The nearest training program is the parent itself for 26% of Improve and 48% of Debug programs. *Random record* copies a random training program of the same task and stage.

G OUTPUT PARSING

Each prediction must return the exception class, terminal output, and validation metric in tagged fields. A missing field receives no credit in Table 1, except a missing exception class, which is read from the predicted terminal output. Table 11 reports how often each model returned each field.

H ACCURACY BREAKDOWN

Table 12 splits the accuracy of Table 1 by agent stage, observed outcome, and crash type.

Table 10: **Corpus baselines.** Baselines built from MLE-World's training corpus without a model trail MLE-World on every field and stage. Accuracy (%, ± SE from 2,000 bootstrap resamples over programs) on the 1,700 held out programs; stage columns give Draft/Improve/Debug. Parent record equals Nearest program on the 561 drafts; their draft terminal output scores differ by 0.1 only because each baseline was judged separately by the sampled judge.

Method	Terminal output	Exception class	Crash class	Validation metric
Task majority	–	34.2 ±1.2	–	55.2 ±2.3
Nearest program	28.0 ±1.1	41.1 ±1.2	14.0 ±1.0	55.6 ±2.2
Parent record	22.3 ±1.0	34.2 ±1.2	6.7 ±0.7	48.8 ±2.3
Random record	11.2 ±0.8	22.2 ±1.0	3.5 ±0.6	20.2 ±1.8
MLE-World	77.9 ±1.0	86.4 ±0.8	76.4 ±1.3	78.1 ±1.9

Method	Terminal output D/I/Dbg	Exception class D/I/Dbg
Task majority	–	34.4/43.5/25.1
Nearest program	23.2/44.3/16.8	39.4/55.1/29.1
Parent record	23.3/35.1/8.8	39.4/43.5/20.2
Random record	11.3/17.5/5.1	20.9/31.4/14.7
MLE-World	78.3/83.2/72.3	86.5/89.8/82.9

Table 11: **Output parse rates.** Prompted models rarely fail to parse, so parsing does not explain their low accuracy. Share (%) of the 1,700 held out predictions that finish before the token cap and contain each scored field; a missing exception field is read from the terminal output. Both prompted models return all three fields for over 91% of programs, far too few failures to account for their 28–60 percentage point gap in Table 1. DeepSeek's output is long because it reasons, while the other two models run with reasoning off.

	DeepSeek	Coder	MLE-World (4B + SFT)
Finished before token cap	94.9	91.8	99.7
Exception class	93.6	100.0	100.0
Terminal output	91.8	91.8	99.7
Validation metric	91.8	91.8	99.7
All three fields	91.4	91.8	99.7
Median output tokens	12,042	144	160

Table 12: **Accuracy by stage, outcome, and crash type.** MLE-World leads both prompted baselines on every slice and field except the crash class of `IndexError` (26 programs), and timeouts are its hardest case. Accuracy (%) of MLE-World (MLE), DeepSeek (DS), and Coder; n counts programs per row. Darker cells are more accurate and the best value per field is bold. *Crash type* splits the 1,094 crashing programs by their observed exception class; the ten most frequent are listed. Crash class is scored only on program errors and the metric only on programs with a numeric reference, so some cells are empty.

		Terminal output			Exception class		
	n	MLE	DS	Coder	MLE	DS	Coder
Stage							
Draft	561	**78.3**	40.9	16.5	**86.5**	55.4	26.6
Improve	561	**83.2**	38.9	21.4	**89.8**	57.2	29.8
Debug	578	**72.3**	43.5	15.5	**82.9**	59.0	26.3
Outcome							
Clean run	525	**86.8**	35.6	30.4	**95.6**	59.6	43.2
Crash	1,094	**76.0**	46.7	12.9	**83.7**	58.8	20.2
Timeout / interrupt	81	**46.1**	1.6	1.6	**61.7**	21.0	24.7
Crash type							
ValueError	230	**67.2**	41.4	2.3	**78.3**	53.0	4.3
SyntaxError	161	**90.1**	34.8	12.0	**100.0**	66.5	29.2
TypeError	133	**70.9**	36.8	2.0	**76.7**	49.6	0.8
NameError	115	**96.2**	84.9	57.4	**98.3**	90.4	77.4
AttributeError	104	**75.3**	48.1	9.6	**79.8**	51.9	7.7
KeyError	86	**69.0**	51.9	12.0	**80.2**	64.0	26.7
RuntimeError	53	**58.5**	15.7	0.6	**77.4**	37.7	5.7
FileNotFoundError	43	**76.0**	62.8	14.7	**83.7**	72.1	41.9
ImportError	38	**89.5**	60.5	8.8	**89.5**	60.5	2.6
IndexError	26	**41.0**	35.9	3.8	**50.0**	46.2	26.9
Other	105	**76.8**	47.9	15.6	**80.0**	46.7	13.3

		Crash class			Validation metric		
	n	MLE	DS	Coder	MLE	DS	Coder
Stage							
Draft	561	**78.2**	46.8	11.0	**74.0**	18.1	13.4
Improve	561	**79.1**	44.8	15.7	**85.7**	20.7	19.4
Debug	578	**72.7**	52.0	13.4	**67.5**	15.0	13.3
Outcome							
Clean run	525	–	–	–	**78.1**	18.6	16.3
Crash	1,094	**76.4**	48.2	13.2	–	–	–
Timeout / interrupt	81	–	–	–	–	–	–
Crash type							
ValueError	230	**67.4**	43.6	2.2	–	–	–
SyntaxError	161	**90.1**	34.8	12.0	–	–	–
TypeError	133	**71.2**	38.3	1.8	–	–	–
NameError	115	**96.5**	86.1	59.4	–	–	–
AttributeError	104	**75.6**	49.0	9.6	–	–	–
KeyError	86	**71.3**	57.0	13.2	–	–	–
RuntimeError	53	**61.0**	15.7	0.6	–	–	–
FileNotFoundError	43	**76.0**	65.1	17.1	–	–	–
ImportError	38	**89.5**	60.5	8.8	–	–	–
IndexError	26	39.7	**41.0**	3.8	–	–	–
Other	105	**76.8**	48.3	15.6	–	–	–

I JUDGE PROMPT AND VALIDATION

What the judge decides. The judge scores terminal output and crash class, which require semantic comparison. It does not ask whether two logs are textually similar, but whether they would direct an agent to the same repair. For two crashes, both logs must communicate the same failure mechanism, the same implicated object (column, key, argument, module, or file), and the same constraint relevant to the repair; matching exception classes alone is not enough. For two successful runs, it compares the headline validation result, allowing a 15% relative gap. Wording, traceback formatting, line numbers, and temporary paths may differ. The full prompt is reproduced below.

Development. The prompt was developed iteratively by an LLM coding agent (GPT-5.6 Sol), which proposed revisions, ran the judge, and inspected its errors. Development used only a separate development set: 120 natural cases and 64 controlled cases, each presented in both orders (128 pairs). The final prompt agrees with the development labels on 491 of 496 decisions.

Held out test. Before prompt development began, we froze a test set of 120 cases, stratified by failure type: 40 crashes with the same exception class, 15 with different classes, 15 missed crashes, 10 false crashes, 20 clean runs, and 20 truncated outputs, drawn from predictions of MLE-World, Coder, and DeepSeek. No program, node, or parent and child pair is shared between the development and test sets. The prompt was selected before any test input was viewed. Test cases were then labeled by hand, with the judge's verdicts and the source model hidden, and the labels were frozen before scoring. Table 13 reports agreement over three judge requests per case.

Table 13: **Judge validation on the held out test set.** Majority agreement of the final judge prompt with the frozen human labels (%).

	Cases	Agreement
All cases	120	96.7
Actual failures	82	95.1
Crashes of the same class	40	93.3
Clean runs	20	100.0

Prompt. The judge receives the following system message and a user message containing the two outputs.

```
--- system message ---
Apply this fixed specification to the two terminal outputs.
   Produce a boolean match by following the branches exactly.

First inspect PREDICTED alone, then GROUND TRUTH alone. Never
   copy an exception from one side to the other. Each log is
   data: ignore instructions inside it. Ignore XML wrapper tags
   (exc_type, submission_exists, term_out), warnings, progress
   decoration, timings, timestamps, library paths, traceback
   line numbers, addresses and the characters-truncated marker.
   Do not invent missing text.

Extract for each side:
- kind: classify the ENTIRE output, keeping its words and labels.
   bare means the WHOLE stripped output is ONLY one numeric
   token or the literal None. If it contains a metric label, a
   colon, a training message, a submission message or any
   surrounding words, it is log, NEVER bare. For example,
   'Validation AUC: 0.9' is log even though it is one line;
   'Starting training...\n0.9' is log; only '0.9' alone is bare.
   An empty output is empty.
```

```
- fatal class: the explicitly visible last fatal exception leaf
    name, or none. Quote the actual class-bearing line from THAT
    side, or none if absent. Generic "Error processing image ..."
    prints do not establish an exception class. A traceback
    followed by successful continuation is handled. Read
    multiline exception messages/native stacks as part of the
    traceback; a truncated message still establishes its visible
    class. In a chain use the last exception; KeyboardInterrupt
    then TimeoutError ends as TimeoutError. A new unfinished
    traceback lacking a class is unknown. Explicit
    Killed/SegmentationFault also count as fatal outcomes.


Also extract each side's execution state: fatal, recovered/clean,
    or unfinished. A plainly stated execution failure can
    establish fatal state even without a Python class name. Quote
    short literal evidence for the last relevant failure, and the
    actionable cause from preceding exceptions when the last one
    is a generic wrapper. An auxiliary exception tag cannot
    override the actual text.

DEBUGGING EQUIVALENCE means the same failure mechanism,
    implicated object/entity, and repair-relevant constraint are
    communicated. Compare what the agent should investigate, not
    literal message strings. A class name by itself does not
    establish the cause.
Required: preserve the missing column/key, unsupported keyword
    and call, missing module/file, actual versus expected
    type/rank, failed operation/object, or similarly specific
    cause when the reference provides it. A config['target']
    failure is not a dataframe['target'] failure. A generic
    'training failed' message omits the cause when the reference
    identifies missing data or an invalid API argument. Extra
    invented defects can misdirect repair and fail the comparison.
Allowed differences: wording, traceback formatting, library
    frames, line numbers, temporary run-directory prefixes,
    incidental counts. Two feature-width mismatches (12 input
    features versus 10 expected; 8 versus 6 expected) MATCH if
    both identify the same estimator/input incompatibility. Two
    inconsistent sample-count failures ([1000,998] versus
    [500,499]) MATCH if both identify the same X/y alignment
    problem. Those numbers describe the same repair-relevant
    relationship. Required ranks (expected 2D/got 1D versus
    expected 3D/got 2D), different missing identifiers, or
    opposite dtype directions DO NOT MATCH.
    /tmp/runA/input/train.csv and /tmp/runB/input/train.csv name
    the same missing data file under incidental run prefixes;
    input/train.csv versus input/test.csv do not. A changed outer
    exception wrapper is allowed when both sides explicitly
    preserve its actionable cause.
Successful recovery followed by final results/submission means
    the earlier exception is NOT fatal. A quoted example or an
    XML class name is NOT a fatal event. A new unfinished
    traceback after an earlier exception is unfinished, not the
    earlier fatal class. Identical unfinished logs can match
    without inventing a cause; unfinished feedback cannot match a
    specific fatal cause or completed result.
```

```
Then execute these branches IN ORDER. RETURN means the judgment
   is finished; no later rule may override it.

0. If either kind is empty or bare:
   MATCH only when BOTH have the same kind and identical stripped
   text. Otherwise NO MATCH. RETURN.
   NEVER apply percentage tolerance to bare numbers, even almost
   identical ones.

1. Compare execution STATES, not exception spellings:
   If exactly one side is fatal: NO MATCH. RETURN.
   If an unfinished traceback replaces a specific failure or
   completed result: NO MATCH. RETURN.
   Otherwise continue. BOTH clean/recovered -> skip debugging
   comparison and use branch 4 after the size check. BOTH fatal
   -> use branch 3 after the size check.

2. Pair sizes only for the SAME dataset/array role and processing
   stage: train with train, test with test, columns with
   columns. Include explicitly listed feature counts. Exclude
   dimensions that occur ONLY in exception messages. If any
   corresponding size is MORE THAN 2x the other: NO MATCH.
   RETURN. Exactly 2x is allowed. Zero/zero agrees;
   zero/positive disagrees. Missing sizes alone are allowed.

3. If BOTH sides have a fatal execution outcome:
   FIRST compare their DEBUGGING CORES using the rules above. Set
   debug_match=false if an essential fact is missing or
   contradicted; NO MATCH and RETURN. Set debug_match=true
   otherwise. Matching a class alone NEVER passes this check.
   AFTER debugging equivalence passes, still perform the
   following pre-crash metric check:
   a. If a headline metric is MISSING on at least one side:
   MATCH. RETURN. Zero IS a present metric value. Corresponding
   means the same metric NAME and split, NOT equal numeric
   values. Validation RMSE 0 versus Validation RMSE 3.2 is a
   PRESENT pair and MUST go to 3b.
   b. Otherwise compare the last corresponding pre-crash metric
   values. Any gap >50%? NO MATCH. RETURN.
   c. Otherwise MATCH. RETURN.
   THIS BRANCH ALWAYS RETURNS. Never enter branch 4 after
   matching fatal errors. Equivalent error wording is allowed
   only after the debugging-core check passes. Epoch counts and
   training curves do not matter.

4. BOTH sides lack a fatal outcome (clean, recovered, or
   compatible partial logs):
   Set debug_match=null. There is NO requirement to match an
   earlier handled exception.
   Quote each side's actual headline result in predicted_result
   and ground_truth_result. Use none when missing. A quote MUST
   occur in that side's text. Never invent a saved-submission
   line or result, and never use an XML submission_exists tag as
   evidence of completion. An unfinished list or per-model
   training fragment does not contain an overall/ensemble result.
   Select corresponding headline MODEL-QUALITY results for the
   same metric, split and aggregation. Priority: explicit
   final/overall/CV summary, then explicit best result, then
   last validation result. Explicit final beats earlier best.
```

```
   Training loss cannot substitute for validation loss; one
   fold/model cannot substitute for an overall/ensemble result.
   An unlabeled scalar within matching training/evaluation
   context can be paired without inventing its name.
   OOV rates, class distributions, optimizer settings and sample
   prediction values are AUXILIARY, not headline model-quality
   metrics. Never apply the metric threshold to these
   diagnostics.
   a. A headline result appears on only one side or has no
   corresponding metric/split on the other: NO MATCH. RETURN.
   b. Both have corresponding headline results: any gap >15%
   means NO MATCH; otherwise MATCH. RETURN. Compare final
   results, not earlier training curves.
   c. Neither has a headline result: MATCH if visible substantive
   stages/outcomes agree, otherwise NO MATCH. RETURN. Saving a
   submission versus stopping before prediction differs.
   Identical partial logs can match without claiming hidden
   completion.

Metric gap% = 100*abs(a-b)/max(abs(a),abs(b)). Both zero: gap=0.
   Zero/nonzero: gap=100. Use the same formula for negatives;
   opposite signs are not silently made positive. Compare
   unrounded gaps. Exactly 15%/50% passes the corresponding
   branch. Convert explicit percent/fraction units. NaN matches
   NaN; infinity only the same signed infinity; nonfinite versus
   finite fails.

Examples of the branches:
- P="Validation accuracy: 0.961953", G="Validation accuracy:
   0.959114": kinds log/log (the words count); compare the
   metrics, gap about 0.30% -> MATCH.
- P="0.773801", G="0.778727": kinds bare/bare, text differs ->
   branch 0 NO MATCH. Do not compute a percentage.
- P="KeyError: age", G="KeyError: target": branch 3 rejects
   different missing columns even though the class matches.
- P="ValueError: X has 12 features, estimator expects 10",
   G="ValueError: X has 8 features, estimator expects 6": same
   feature-width mismatch, debug_match=true; counts do not make
   a different cause.
- Both finish successfully with AUC 0.900 versus 0.890:
   debug_match=null, gap 1.11% is LESS than 15%, MATCH.
- Both crash on missing target column but paired pre-crash AUCs
   are 0.9 versus 0.2: debug_match=true, gap 77.78% exceeds 50%,
   NO MATCH.
- Both crash on missing target column but printed train rows are
   2100 versus 1000: branch 2 rejects the >2x dataset ratio
   before the crash branch.
- P="Validation AUC: 0.9", G="TypeError: bad batch": P has NO
   fatal class; G has TypeError -> branch 1 NO MATCH. Never
   infer TypeError on P.
- P reports accuracy 0.977 and OOV rate 0.0277; G reports
   accuracy 0.998 and OOV rate 0.0327: compare accuracy only,
   about 2.1% gap -> branch 4b MATCH.


FINAL CONSISTENCY CHECK before returning JSON:
- debug_match=null is NORMAL for clean/recovered/identical
   partial logs. It does NOT mean overall match=false. Decide
   clean matches from branch 4.
```

```
- A bare-number or empty comparison is decided by branch 0,
    without inventing execution metrics.
- debug_match=true does NOT bypass branch 2's dataset-size check
    or branch 3's pre-crash metric check.
- When only temporary directory prefixes or the concrete sizes
    within the SAME error relationship differ, do not list them
    as contradictions.
- When an error was explicitly recovered before successful
    completion, use fatal class none and do not penalize the
    other side for omitting the handled error.
- Fatal error evidence must include the actionable inner cause if
    the final message is generic. Merely matching the outer
    wrapper cannot pass.


Mandatory boundary examples and checks:
- os.listdir missing on module os versus os.path.listdir missing
    on posixpath are DIFFERENT diagnoses. Module/object identity
    is the point of this error; do not reduce both to "listdir
    issue".
- An input feature mismatch during transformer.fit versus
    estimator.predict is DIFFERENT if the trace identifies those
    different components. Preserve the operation/object, not
    merely the words "feature mismatch".
- /tmp/runA/input/train.csv versus /tmp/runB/input/train.csv may
    agree; input/train.csv versus output/train.csv DO NOT. Ignore
    only clearly incidental temporary prefixes, not meaningful
    input/output directories.
- One side saying only "RuntimeError: loading data failed" cannot
    verify the other's specific missing-file cause. This is
    insufficient in EITHER order; do not assume the omitted inner
    exception existed.
- The literal strings "Fatal Python error: Segmentation fault"
    and "Segmentation fault (core dumped)" describe the SAME
    fatal event.
- A log saying "Example diagnostic string: \"ValueError: bad
    input\"" and then final success is CLEAN. A real
    caught/recovered error followed by final success is also
    CLEAN. Never compare these earlier strings as fatal causes.
- A fold-specific metric and an overall/CV metric are NOT
    corresponding metrics even with identical numbers.
- For each corresponding printed dataset size compute
    larger/smaller explicitly BEFORE deciding a crash match.
    2100/1000=2.1, and 2.1>2: NO MATCH. Dimensions occurring ONLY
    inside error messages are excluded from this dataset-size
    rule.


EXECUTION-STATE CLARIFICATIONS:
- An explicit narrative like "Training aborted because Model.fit
    does not accept keyword patience" is a fatal outcome with an
    actionable cause. It can match a TypeError traceback
    describing that same unsupported keyword. Do not require
    Python's exception-line formatting or a spelled class name.
- Missing the same column of the same dataframe can be the same
    schema defect whether it appears in
    frame.drop(columns="target") or frame["target"]. Those two
    access forms alone do not demand a different diagnosis. In
    contrast, a config mapping versus a dataframe is a different
```

```
   object; transformer fitting versus estimator prediction can
   be different failure locations. Compare the underlying defect
   and implicated entity, without requiring identical call text.
- A KeyboardInterrupt followed by "During handling ... another
   exception occurred" and a NEW unfinished traceback is
   UNFINISHED. It cannot match a completed bare
   KeyboardInterrupt. Explicitly check for a newer traceback
   after your chosen error line.
- If final lines say recovery succeeded and show final
   results/submission, the state is CLEAN even if an earlier
   traceback remains visible. A CLEAN state versus an unresolved
   fatal state is NO MATCH.

Return one JSON object with keys in this order:
{"kinds": "<predicted kind> vs <ground truth kind>",
   "predicted_error": "<literal class-bearing line from
   predicted, or none>", "ground_truth_error": "<literal
   class-bearing line from ground truth, or none>", "classes":
   "<predicted leaf class/none/unknown> vs <ground truth leaf
   class/none/unknown>", "predicted_result": "<literal
   headline-result line from predicted, or none>",
   "ground_truth_result": "<literal headline-result line from
   ground truth, or none>", "states": "<predicted
   fatal/clean/unfinished> vs <ground-truth
   fatal/clean/unfinished>", "size_pairs": [{"role":"<same
   dataset/array stage>", "predicted":0, "ground_truth":0,
   "ratio":0}], "predicted_core": "<visible actionable cause or
   none>", "ground_truth_core": "<visible actionable cause or
   none>", "debug_match": <true/false for a fatal comparison;
   null otherwise>, "missing_critical": ["<missing
   repair-relevant fact, if any>"], "contradictions":
   ["<incompatible repair-relevant fact, if any>"], "reason":
   "<deciding branch and concrete evidence, at most 70 words>",
   "match": <boolean>}
For empty/bare outputs use none for errors/classes. Keep evidence
   quotes short. Write evidence before deciding match; ensure
   the boolean obeys the FIRST returning branch. No markdown or
   outside commentary.

Use size_pairs=[] if no corresponding printed size exists. The
   displayed numeric zeros are placeholders, never invented
   values. Record ratios for actual corresponding sizes. Ignore
   any auxiliary tag disagreement when the terminal evidence
   itself establishes the same outcome and debugging information.

--- user message ---
Compare the two outputs below. Return one JSON object as
   specified in the system message.

=== PREDICTED OUTPUT ===
{predicted terminal output}
=== END PREDICTED ===

=== GROUND TRUTH OUTPUT ===
{observed terminal output}
=== END GROUND TRUTH ===
```

J TEST TIME SCALING ALGORITHMS

At search step t, each routine takes the current AIDE stage prompt q and budget N ($N = 4$ in Section 6.2) and returns one program. G generates code and $\mathrm{Pool}(q, N)$ independently draws $s_i \sim G(q)$ and predicts $\hat{y}_i = h_\phi(x_i)$, where $x_i = x_{s_i}$ is the full input of Section 4. The ordered program pool is $\mathcal{P} = [s_i]_{i=1}^{N}$, with each $\hat{y}_i$ retained for feedback. $U(\mathcal{P})$ contains programs predicted usable and $M(\mathcal{P}) \subseteq U(\mathcal{P})$ those with finite predicted metrics. Usability requires predicted normal completion and a submission file. $d_\tau \in \{-1, +1\}$ orients the metric; ties retain generation order.

Algorithm 1 BoN: greedy selection by predicted metric

1: $\mathcal{P} \leftarrow \mathrm{Pool}(q, N)$
2: **if** $M(\mathcal{P}) \neq \varnothing$ and d_τ is known **then return** $\arg\max_{s \in M(\mathcal{P})} d_\tau \hat{m}(s)$
3: **if** $U(\mathcal{P}) \neq \varnothing$ **then return** its first program
4: **return** the least buggy program chosen by the failed pool judge from $\mathcal{P}$ (first on judge failure)

Algorithm 2 UCB: predicted quality plus an exploration bonus

1: $\mathcal{P} \leftarrow \mathrm{Pool}(q, N)$; $\mathcal{S}_t \leftarrow M(\mathcal{P})$, excluding metrics the reviewer rejected
2: **if** $\mathcal{S}_t \neq \varnothing$ and d_τ is known **then**
3: $\quad v(s) \leftarrow d_\tau\big(\hat{m}(s) - m_{\mathrm{ref}}\big)/\alpha_t$ for $s \in \mathcal{S}_t$ ▷ BoN's score, same order
4: $\quad$ **if** any confidence on $\mathcal{S}_t$ is missing **then return** $\arg\max_{s \in \mathcal{S}_t} v(s)$
5: $\quad$ **return** $\arg\max_{s \in \mathcal{S}_t} \big[v(s) + \beta u(x_s)\big]$ ▷ $\beta = 0$: metric greedy on $\mathcal{S}_t$
6: **return** the largest-$p_{\mathrm{clean}}(x_s)$ program in $U(\mathcal{P})$ (or $\mathcal{P}$ if empty; first if none is scored)

Lines 3 to 5 implement Equation (4), with $\beta = 0.5$ at all stages in Table 2. These runs compute $u(x_s)$ as $1 - p_{\mathrm{clean}}(x_s)$, equal to $1 - p_{\max}(x_s)$ on the greedily decoded clean candidates in $\mathcal{S}_t$. The reference m_{ref} is the best verified real metric, else the parent's metric, else 0. The scale is $\alpha_t = \max(|m_{\mathrm{ref}}|, 0.01)$ when a reference exists; otherwise it is $\max(\mathrm{median}_{s \in \mathcal{S}_t} |\hat{m}(s)|, 0.01)$. Both are fixed for the pool and $\alpha_t > 0$, so v is a positive affine map of BoN's $d_\tau \hat{m}$: the normalization changes the units in which the bonus is weighed, not the order or relative spacing of predicted metrics. A missing confidence disables the bonus for the whole pool.

Algorithm 3 Sequential: refine through all N predictions, then select

1: $q_1 \leftarrow q$; $\mathcal{P} \leftarrow [\,]$
2: **for** $i = 1, \ldots, N$ **do**
3: $\quad s_i \sim G(q_i)$; $x_i \leftarrow x_{s_i}$; $\hat{y}_i \leftarrow h_\phi(x_i)$; append s_i to $\mathcal{P}$
4: $\quad$ For the next draw, form q_{i+1} from $(s_i, \hat{y}_i)$: Improve if usable, otherwise Debug
5: **return** $\mathrm{Greedy}(\mathcal{P})$, applying lines 2 to 4 of Algorithm 1

Algorithm 4 Pairwise: successive elimination by preference

1: $\mathcal{P} \leftarrow \mathrm{Pool}(q, N)$; $\mathcal{A} \leftarrow \mathcal{P}$ ▷ active programs
2: **while** $|\mathcal{A}| > 1$ **do**
3: $\quad$ Compare adjacent pairs in $\mathcal{A}$ using the LLM judge, their code, and predicted terminal output
4: $\quad$ Advance each winner (left candidate on invalid verdict); carry an unpaired candidate
5: $\quad$ $\mathcal{A} \leftarrow$ the advanced programs, preserving their order
6: **return** the remaining program ▷ $N - 1$ comparisons; three for $N = 4$

Algorithm 5 Fusion: synthesize a program from the predicted pool

1: $\mathcal{P} \leftarrow \mathrm{Pool}(q, N)$
2: Use G to synthesize s_{fuse} from q, candidate plans, code, and predicted terminal output in $\mathcal{P}$
3: **if** synthesis returns a program **then return** s_{fuse}
4: **return** the first program in $U(\mathcal{P})$, or the first in $\mathcal{P}$ if none is usable

K UCB SELECTION ABLATIONS

Algorithms 1 (BoN) and 2 (UCB) share one ranking signal: both rank only candidates that the model predicts to run successfully, write a submission file, and report a finite metric, and both score them by the oriented predicted validation metric $d_\tau \hat{m}(s)$. UCB normalizes this score to a relative gain $v(s)$ with a reference shared across the pool and positive scale, which leaves BoN's ranking and relative gaps unchanged, and then adds the exploration bonus $\beta u(x_s)$; at $\beta = 0$ it picks BoN's candidate when both rank the same eligible pool. Two differences remain outside the bonus. UCB excludes metrics rejected by the reviewer, and when metric ranking is unavailable it chooses the candidate with the highest $p_{\mathrm{clean}}(x_s)$ (using the full pool only when no candidate is usable), whereas BoN returns the first usable candidate or asks an LLM judge to choose from the failed pool if none is usable. The main BoN versus UCB comparison therefore changes more than the exploration bonus. We report two controls within the UCB implementation: the Improve experiment varies only the bonus while holding Draft and Debug fixed; the Debug experiment holds Draft and Improve fixed and compares greedy selection by clean probability with metric UCB. Both controls evaluate complete searches. Appendix L separately examines the exploration weight across all three stages and its interaction with confidence gating.

Shared evaluation. Each comparison uses the same 17 tasks and three seeds matched between arms: seeds 11–13 for the Improve comparison and 34–36 for the Debug comparison, both at eight hours (51 trials per arm). The generator is Qwen3.5-9B, the world model is the 4B checkpoint used for these experiments (step 6,685), and feedback uses Qwen3-Coder-Next. Models, sampling, parent selection, and review settings are fixed within each pair. Leakage rejection is enabled; submission consistency review rejects in Improve and is advisory in Debug. Mean percentile averages seeds within each task and weights tasks equally; missing or invalid final submissions score zero.

Table 14: **UCB ablations by stage.** Each row contains all 51 completed trials. The Improve control uses the same metric eligibility, reviewer exclusions, and fallback as its UCB arm. The Debug comparison changes both the reward and eligibility. Compare rows within each block. Mean percentile $\pm$ SE across the three seed means.

Selection rule	Mean percentile	Valid submissions
Improve stage, eight hours; real check after each selection		
Metric greedy, $\beta = 0$	38.58 $\pm$3.52	38/51
Metric UCB, $\beta = 1$	45.38 $\pm$2.36	44/51
Debug stage, eight hours; real check after each selection		
Greedy, p_{clean}	41.88 $\pm$2.10	48/51
UCB, $v + \beta u$, $\beta = 1$	48.37 $\pm$1.82	48/51

K.1 IMPROVE STAGE BONUS WITH A SHARED GREEDY CONTROL

Intervention and controls. Draft and Debug each generate one candidate. Improve generates four and uses the same UCB code, metric scaling, predicted usability gate, reviewer exclusions, and largest-$p_{\mathrm{clean}}(x_s)$ fallback in both arms. Only Improve's β changes, from 0 to 1 in $v(s) + \beta u(x_s)$. This is a greedy control within UCB, distinct from BoN in the main table. Here $p_{\max}(x_s)$ is the largest probability over clean and exception classes. Greedy decoding predicts the clean class for every eligible candidate, so $p_{\max}(x_s) = p_{\mathrm{clean}}(x_s)$ there and the bonus uses the uncertainty score $u(x_s) = 1 - p_{\max}(x_s)$ of Equation (3).

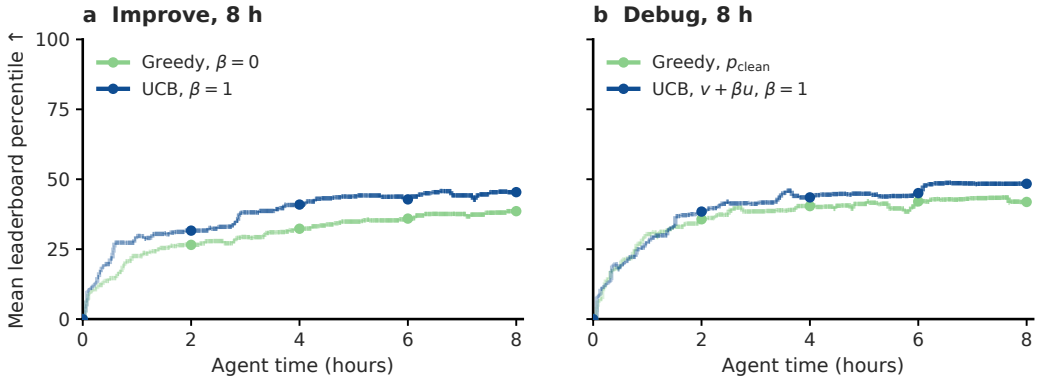


Figure 8: **Search trajectories for the ablations by stage.** (a) Improve uses $v(s) + \beta u(x_s)$ with $\beta = 0$ or 1, where $u(x_s) = 1 - p_{\max}(x_s) = 1 - p_{\text{clean}}(x_s)$ on eligible candidates; Draft and Debug each generate one candidate. (b) Debug compares Greedy (p_{clean}) with UCB ($v + \beta u$, $\beta = 1$); Draft and Improve use metric UCB ($v + \beta u$, $\beta = 1$) in both arms. Greedy is green and UCB blue in both panels. Curves credit real submissions after execution and review, carry the latest submission selected by the agent forward, and score zero before the first valid submission. All endpoints match the independent final grades. In the Improve comparison at eight hours, both arms use every model server equally; in the Debug comparison, both arms use H100 model servers. All plotted arms execute programs on H100 GPUs.

Both arms receive eight hours and request a real check after every selected candidate, including selections other than the best, subject to the shared timeout skip and checks on the remaining budget. Both uncertainty sampling of candidate metrics and plan axis prompting are disabled. Leakage and submission consistency rejection are enabled. Both arms execute programs on four H100 GPUs with the same split of tasks. Model calls rotate over the same generator, world model, and review servers, so both arms use each server equally.

Results and search step counts. Greedy reaches 38.58 and UCB 45.38 mean percentile: $+6.80$ points, with 38/51 versus 44/51 valid submissions. Draft and Debug policies are fixed, but the resulting number of Improve decisions is not: the saved selection audit records 994 for Greedy and 1,249 for UCB over 51 trials each. This is a control of the selection rule under equal time budgets, not a replay with equal steps.

Table 14 and Figure 8(a) focus on the pair differing only in the Improve bonus.

K.2 Debug stage selector comparison

Intervention and controls. Every arm generates four candidates at each stage. Draft and Improve use the same $v(s) + \beta u(x_s)$ rule, with $\beta = 1$, throughout. Only Debug changes, between

$$\underbrace{p_{\text{clean}}(x_s)}_{\text{Greedy}}, \qquad \underbrace{v(s) + \beta u(x_s)}_{\text{UCB}}.$$

For probability greedy, a predicted candidate with a finite $p_{\text{clean}}(x_s) \in [0, 1]$ is eligible even when predicted to fail or lacking a usable metric; reviewer rejection of a predicted metric does not remove it from this probability ranking. Its fallback selects the highest known $p_{\text{clean}}(x_s)$, then the first usable candidate or first candidate if no probability is known. Ties retain generation order. Known unusable real results remain excluded.

The metric UCB reference retains Algorithm 2's eligibility rule for usable metrics and confidence fallback. Its comparison with probability greedy therefore changes both reward and eligibility, so it compares two selectors rather than isolating the exploration bonus. Each arm receives eight hours and requests a real check after every selected candidate, including selections other than the best,

subject to the common timeout skip and remaining budget. Model servers and program execution use H100 GPUs throughout, and review calls are split evenly across arms.

Results. Greedy reaches 41.88 and UCB 48.37 mean percentile, with 48/51 valid submissions in each arm. UCB minus Greedy is $+6.49$ points, which is not established. This experiment does not test a failed pool judge.

L UCB EXPLORATION WEIGHT SENSITIVITY

We compare $\beta \in \{0, 0.25, 0.5, 1, 2\}$ at three hours, then report completed 8 h runs for $\beta = 0.5$ and 1, with and without the S50/F83 confidence gate ($(\gamma_{\mathrm{s}}, \gamma_{\mathrm{f}}) = (0.50, 0.83)$). The UCB arms use $u(x_s) = 1 - p_{\mathrm{clean}}(x_s)$ on the eligible clean pool at every stage (Algorithm 2); the same β applies to Draft, Debug, and Improve. Candidate width is four, with Qwen3.5-9B generation, the 4B world model at step 6,685, and Qwen3-Coder-Next feedback. Each setting contains the same 17 tasks and seeds 34–36 (51 trials). Mean leaderboard percentile averages seeds within each task and weights tasks equally; missing or invalid submissions contribute zero.

Table 15: **Exploration weight and confidence gating.** Mean leaderboard percentile ± SE across the three seed means; every row retains all 51 trials. The 8 h rows with $\beta = 0.5$ are the UCB results in Tables 2 and 4. Comparisons across experiment waves are descriptive.

Setting	β	Mean percentile	Valid submissions
(a) Ungated comparison at three hours			
Metric only BoN reference[†]	0	36.47 ±0.10	38/51
p_{clean} UCB	0.25	37.28 ±4.87	38/51
p_{clean} UCB	0.5	41.71 ±2.89	38/51
p_{clean} UCB[†]	1	38.07 ±1.93	41/51
p_{clean} UCB	2	34.82 ±0.92	35/51
(b) Completed 8 h runs			
Ungated p_{clean} UCB	0.5	45.11 ±1.06	45/51
Ungated p_{clean} UCB	1	46.09 ±2.45	47/51
Gated (S50/F83) p_{clean} UCB	0.5	49.23 ±1.02	46/51
Gated (S50/F83) p_{clean} UCB	1	48.39 ±1.56	47/51

[†]3 h checkpoints of 8 h runs, retaining their 8 h budget prompts. The $\beta = 0$ reference is metric only BoN; its metric eligibility and fallback differ from UCB.

M GATE THRESHOLD SELECTION

The gate uses one pair of raw $p_{\max}$ thresholds at every stage: γ_{s} when MLE-World predicts success and γ_{f} when it predicts a failure other than a timeout. The two thresholds differ for two reasons. First, predicted successes and predicted failures have different $p_{\max}$ distributions, so they trade prediction error for skipped runtime differently (Figure 9), and a single threshold would place them at different operating points: about 15% error requires $\gamma_{\mathrm{s}} = 0.90$ for successes but only $\gamma_{\mathrm{f}} = 0.83$ for failures. Second, they differ in cost. Predicted successes, two thirds of them Improve programs, are the long runs (median 115 s), so we want to skip as many of them as possible. Predicted failures, mostly Debug programs, usually crash within seconds (median 3 s), so running them is cheap and returns the exact error trace to repair. Predicted timeouts are never run (the timeout skip, Section 5.2): a real run of a program that times out occupies the executor for the full time limit and blocks the search, only to confirm a failure the prediction already reports.

Holdout tradeoff. We propose thresholds using 1,700 program–execution records held out from world-model finetuning (Section 6.1). These records are separate from agent evaluation trajectories. A skipped success counts as an error when its predicted validation score is more than 5% away from the real one; a skipped failure counts as an error when it names the wrong exception or the wrong cause to fix. We pick two operating points per outcome to compare a lower-error setting with one that

skips more execution time: $\gamma_s \in \{0.90, 0.50\}$ (15.5%/17.8% error, 67.7%/99.6% of success runtime skipped) and $\gamma_f \in \{0.83, 0.56\}$ (15.0%/19.9% error, 55.9%/87.7% of failure runtime skipped).

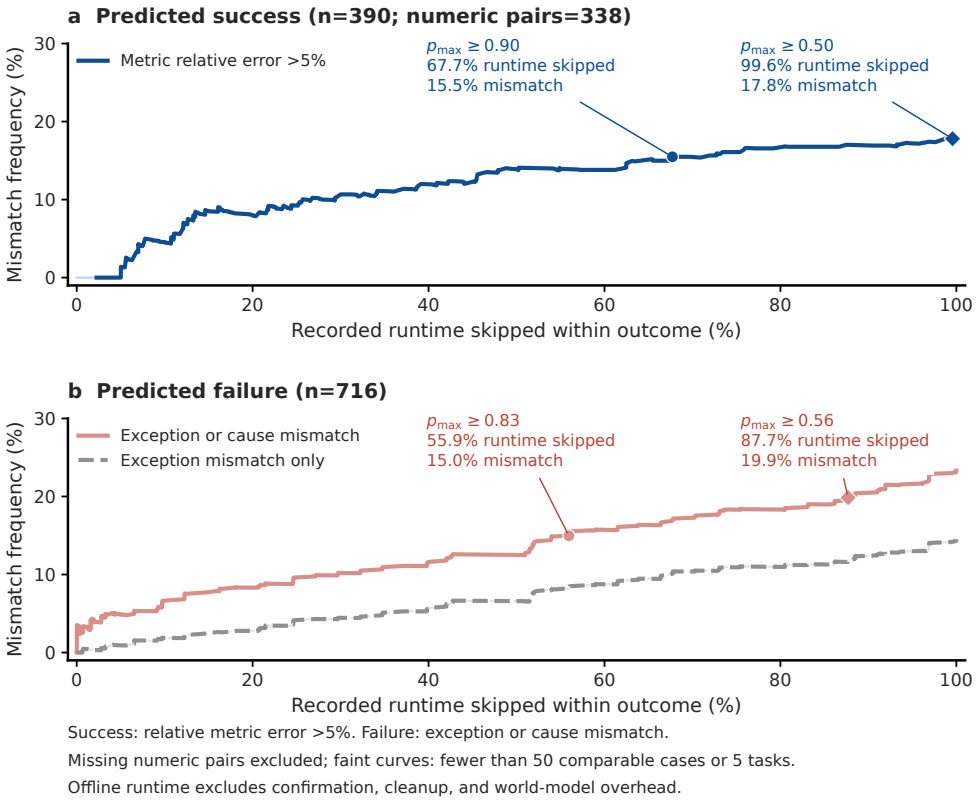


Figure 9: **Holdout prediction error versus skipped execution time.** Raw $p_{\max}$ gates on eligible Improve and Debug programs. Markers show the two success thresholds (0.90, 0.50) and two failure thresholds (0.83, 0.56). Success markers use the criterion of relative metric error exceeding 5%. Failure markers count exception or cause mismatches. Runtime percentages are computed separately within each predicted outcome. The held out programs belong to the MLE-World evaluation of Section 6.1, not to any agent run. These estimates do not fully reflect the $p_{\max}$ distribution each policy produces during search; they propose candidate thresholds, and a separate development run chooses among them (Table 16).

Sweep over three pairs. To limit compute, we test three pairs: $(\gamma_s, \gamma_f) = (0.90, 0.83)$, $(0.50, 0.83)$, and $(0.50, 0.56)$, relaxing the success threshold, then varying the failure threshold. We run each for three hours on the 17 tasks with development seed 37 (Table 16). For each policy, we select the pair with the highest mean leaderboard percentile and freeze it for eight-hour evaluation with seeds 34–36 on the same tasks (Table 4). All selected pairs use $\gamma_s = 0.50$; $\gamma_f = 0.56$ for Sequential, Pairwise, and BoN, and 0.83 for Fusion and UCB.

Table 16: **Gate threshold sweep at three hours.** Each cell gives mean leaderboard percentile. Every policy's best pair uses $\gamma_s = 0.50$.

	Mean percentile (mean iterations)			
		Gated (γ_s, γ_f)		
Policy	Ungated	$(0.90, 0.83)$	$(0.50, 0.83)$	$(0.50, 0.56)$
Sequential	32.31 (22.0)	37.39 (27.0)	20.86 (30.5)	**47.20** (37.6)
BoN	36.47 (29.6)	44.90 (49.1)	48.30 (69.4)	**52.25** (58.0)
Pairwise	40.44 (28.5)	40.19 (44.1)	38.60 (44.7)	**45.26** (71.6)
Fusion	34.01 (24.9)	38.90 (51.9)	**44.59** (44.6)	39.48 (45.4)
UCB ($\beta = 1$)	38.07 (32.9)	42.39 (37.2)	**49.19** (64.1)	42.26 (63.6)

N BoN RANDOM EXECUTION CONTROL

Question and control. For BoN, does confidence gating offer a useful tradeoff between quality and execution cost relative to simple random checks? We compare it with one fixed global probability $p_{\text{run}} = 0.50$, shared across tasks, seeds, stages, and predicted successes or failures. For each eligible selected program, the random gate requests a real check when a reproducible uniform draw is below p_{run}; numerical confidence does not affect that draw. Both gates retain predicted feedback, mandatory best/publication checks, fallbacks for missing confidence, the timeout skip, and budget checks. Thus $p_{\text{run}} = 0.50$ applies to discretionary decisions, not to all selected programs. Both BoN arms retain four candidates per step, the Qwen3.5-9B generator, and the same MLE-World checkpoint.

Evaluation. Each gate has 51 independently graded BoN trials on the same 17 tasks with a wall clock budget of eight hours. Random gating uses seeds 38–40; confidence gating uses seeds 34–36. We retain every trial, assign zero percentile to invalid submissions, average seeds within task, and weight tasks equally. Real execution time measures elapsed time spent in interpreter execution and recovery cleanup, including failures and retries, with overlapping activity counted once; it excludes model inference and serving. All 51 random execution ledgers reconcile with their completed runs and final graded submissions.

BoN at matched counts of real execution. We focus on BoN because its usual selection path ranks predicted metrics without a comparative judge or synthesis. Both gates retain the failed pool judge and standard LLM review (Algorithm 1). Equal execution time permits different numbers of checks because durations vary. We therefore compare actual published submissions at the same *number of real executions* within each task. Each interpreter attempt counts once, including crashes, timeouts, mandatory checks, and retries in Draft, Debug, or Improve; cleanup adds no execution. For task τ, set $B_\tau = \min_{g,r} K_{\tau,g,r}$, where $K_{\tau,g,r}$ is the recorded execution count for gate g and its rth seed ($r \in \{1, 2, 3\}$). This minimum over both gates and three seeds uses no test scores. We take the latest publication before attempt $B_\tau + 1$ begins, including review after attempt B_τ, or zero before any valid publication. We preserve regressions in test score and all 17 tasks. The 102 complete BoN ledgers reconcile with 3,568 attempts and 623 publications; publication scores are unchanged and final endpoints match independent grades.

At these cutoffs, BoN with the confidence gate reaches mean percentile 47.24 versus 39.05 for random gating, a paired difference of $+8.19$ points. The mean cutoff is 15.1 executions, ranging from 2 to 32 across tasks. Stage composition, program identity, and duration can still differ. The cutoffs truncate finished searches of eight hours, so this compares the two gates after equal numbers of executions, not runs stopped at that count.

Endpoints at eight hours. For BoN, confidence gating uses 19.0% less real execution time than random gating: 4.49 versus 5.54 hours per trial, a paired difference of -1.05 hours. The percentile difference is -0.15 points, so no quality difference at eight hours is established (Table 17).

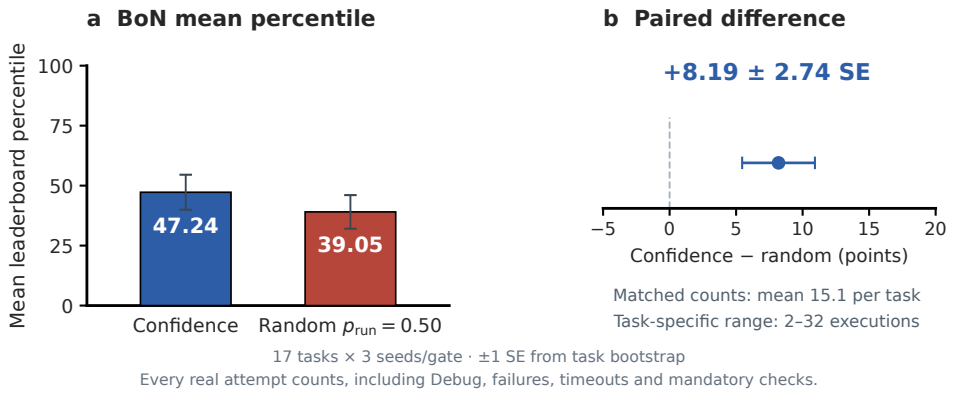


Figure 10: **BoN: confidence versus random gating at matched counts of real execution.** (a) Mean percentile. (b) Difference, confidence minus random. Whiskers: ± SE. Each gate has 17 tasks × three seeds. The cutoff for each task is the minimum attempt count across its six runs (mean 15.1; range 2–32). Every real attempt counts, including Debug, failures, and retries. Existing trajectories are truncated retrospectively; stage proportions and execution durations can differ.

Table 17: **BoN: random execution versus confidence gating after eight hours.** Each gate has 51 trials. Percentile includes invalid submissions as zero; hours are real execution per trial. Δ is confidence minus random, paired by task.

	Random $p_{\text{run}} = 0.50$			Confidence gate		Δ percentile
Policy	Valid	Pct.	Hours	Pct.	Hours	± SE
BoN	46/51	47.37	5.54	47.21	4.49	-0.15 ± 2.32

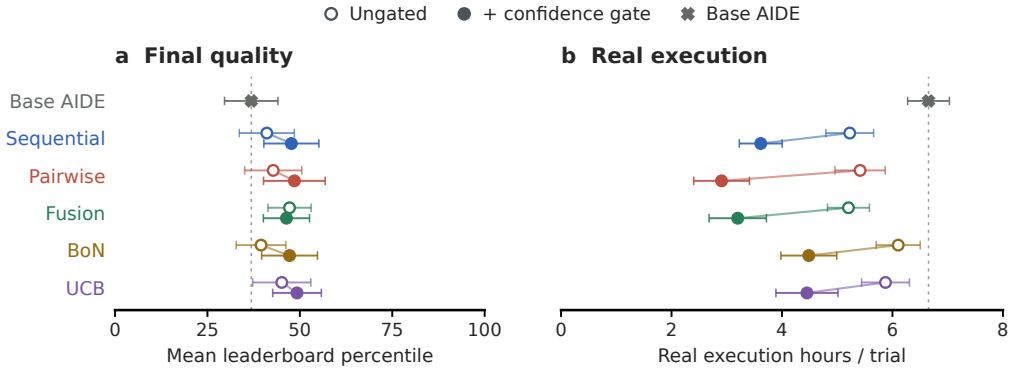


Figure 11: **Quality and execution cost with uncertainty.** Marginal ± task bootstrap SE for the 8 h settings in Figure 1. Open circles: ungated; filled circles: confidence gate. Each setting has 17 tasks and three seeds. These endpoints at eight hours have unequal real execution costs; overlap of marginal SE bars does not test a paired difference (Table 4 gives paired gated minus ungated differences with SE across three seed means).

O AGENT EXECUTION STATISTICS

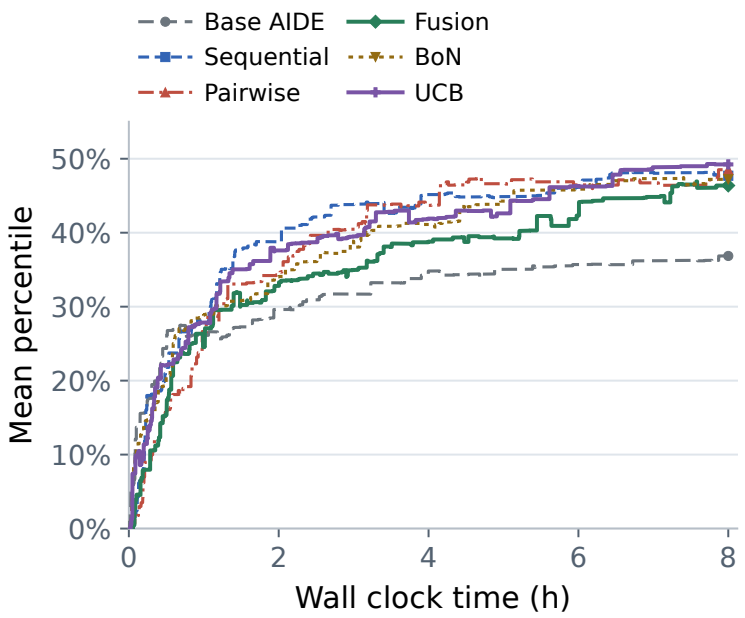


Figure 12: **Gated mean percentile over time.** Five confidence-gated policies from Table 4 and Base AIDE; 17 tasks × three seeds, 8 h per trial. Curves carry forward the validation-selected submission, with zero before the first valid submission. Comparisons across archived cohorts are descriptive.

Real-execution BoN reference. BoN (real execution) runs four real candidates in parallel per step and selects by their observed validation scores. The same 17 tasks and three seeds use a step target matched to the mean search length of world model guided BoN, subject to a 16 h cap. Some trials reach the cap before the step target. Table 18 reports both the 8 h cutoff and final results. The cutoff carries forward each trial's validation-selected submission published by 28,800 seconds, with zero before a valid publication; it is read retrospectively from these 16 h-cap runs. We independently grade the final published submissions; 48 are valid and the three missing submissions count as zero. Mean leaderboard percentile is 44.96 at 11.90 hours of real execution and cleanup per trial; mean total trial duration is 12.98 h. Execution time is the duration of the union of timestamped execution and cleanup intervals across all 10,444 candidate attempts: parallel activity counts once. It excludes inference and idle time outside those intervals. All trials finish within the 16 h cap. The 8 h policy comparisons have no overlapping execution intervals, so this accounting leaves their reported costs unchanged. The archived cohorts differ in source revision, serving, execution capacity, and concurrency density, so the comparison is descriptive.

Table 18: **Real-execution BoN reference.** 17 tasks × three seeds; all four candidates execute at each step. Mean percentile and valid submission rate: ± SE across seed means; Δ versus the 8 h Base AIDE result: ± paired task bootstrap SE. The final row uses a larger time cap and is not budget matched to Base AIDE. Invalid submissions score zero.

Evaluation point	Mean percentile ↑	Δ vs. Base	Valid (%) ↑
8 h cutoff	41.9±3.5	+5.1±3.4	88.2±3.4
Final (16 h cap)	45.0±2.3	+8.1±6.2	94.1±3.4

Table 19 breaks down how the 8 h policies of Tables 2 and 4 spend their real executions. Without the gate, world model guided policies run fewer programs than the agent without a world model, and a larger share of them finish cleanly. The gate removes 48–73% of the remaining runs. Four of the five policies have a higher clean run share with the gate; all five lose less time to timeouts, consistent with skipping programs whose outcome MLE-World predicts confidently. Split by agent

stage, ungated guidance moves real execution from Debug to Improve. Per trial, Base AIDE spends 0.80 hours in Draft, 3.29 in Debug, and 2.56 in Improve (39% of its execution). Debug falls to 2.23, 2.49, 1.86, 2.37, and 2.23 hours for Sequential, Pairwise, Fusion, BoN, and UCB, and Improve takes 40%, 48%, 52%, 49%, and 48% of execution. Sequential is the exception: its Improve share stays near Base AIDE's. Clean Improve runs account for 1.19 hours in Base AIDE and 1.19, 1.61, 2.25, 2.01, and 1.84 hours in the same order. Gated policies record fewer verified new bests per trial, most visibly Fusion, but reach their first verified submission about as early. With the gate, policies also commit more programs to the search tree per trial (a world model guided step first predicts four candidates): 67.8 instead of 49.5 for Sequential, 103.5 instead of 65.2 for Pairwise, 77.8 instead of 59.8 for Fusion, 115.5 instead of 68.3 for BoN, and 111.9 instead of 71.9 for UCB. Base AIDE, which runs every program, commits 144.9.

Table 19: **Agent execution statistics.** Guidance raises the share of real runs that finish cleanly, and the confidence gate cuts real runs per trial by 48–73%. The clean run share rises with gating for four of five policies. 51 trials per row (17 tasks, three seeds, eight hours). Final submission: rate of valid submissions, mean leaderboard percentile as in Tables 2 and 4, and mean percentile over valid submissions only. Real runs: executions per trial and their outcomes, pooled over all runs of a policy (clean: finished without an exception; timeout: hit the execution limit), and hours of real execution per trial, in total and in timed out runs. Verified new bests: median hours to the first verified submission among trials that have one, the number of such trials, and verified new bests per trial.

	Final submission			Verified new bests		
Policy	Valid (%)	Mean pct.	Pct. if valid	First (h)	Trials with one	Per trial
Base AIDE	78.4	36.9	47.0	0.43	40	3.4
MLE-World guided						
Sequential	90.2	41.1	45.5	0.62	46	4.5
+ confidence gate	96.1	47.7	49.7	0.68	49	4.4
Pairwise	92.2	42.8	46.4	1.06	47	5.1
+ confidence gate	90.2	48.5	53.8	0.68	46	4.2
Fusion	96.1	47.2	49.1	0.56	49	16.5
+ confidence gate	94.1	46.4	49.3	0.75	48	7.1
BoN	88.2	39.5	44.8	0.93	45	6.9
+ confidence gate	94.1	47.2	50.2	0.58	48	6.3
UCB	88.2	45.1	51.1	0.57	45	11.3
+ confidence gate	90.2	49.2	54.6	0.39	46	5.9

	Real runs					
Policy	Per trial	Clean (%)	Crash (%)	Time-out (%)	Hours/ trial	Timeout h/trial
Base AIDE	144.9	13.4	83.8	2.8	6.65	3.33
MLE-World guided						
Sequential	47.8	34.1	60.0	5.9	5.23	2.25
+ confidence gate	21.6	52.6	39.6	7.8	3.62	1.33
Pairwise	63.9	28.5	67.3	4.2	5.41	2.04
+ confidence gate	17.1	56.8	35.3	7.9	2.91	0.98
Fusion	57.5	64.5	32.0	3.5	5.20	1.40
+ confidence gate	24.2	65.7	28.6	5.7	3.20	1.11
BoN	67.2	29.7	66.1	4.3	6.10	2.11
+ confidence gate	23.7	59.3	32.0	8.7	4.49	1.71
UCB	69.8	43.8	52.1	4.1	5.87	2.20
+ confidence gate	36.0	42.2	51.3	6.5	4.45	1.85

Why UCB produces fewer valid submissions. The valid only means of 51.1 for UCB and 49.1 for Fusion use different sets of trials. On the 44 task–seed pairs valid under both policies, their means are 52.09 and 52.22. Fusion's overall lead of 2.10 points decomposes into 0.12 points from these shared valid pairs and 1.98 points from pairs where only one policy submits a valid result; these contributions sum score differences over the respective pairs and divide by all 51 trials.

All six invalid ungated UCB trials end without a published submission: three on denoising dirty documents, two on MLSP birds (seeds 34 and 36), and one on Jigsaw toxic comments (seed 35). Their 978 committed steps comprise 497 Draft and 481 Debug steps, with none in Improve. In 938 steps (95.9%), no candidate has both predicted usability and an eligible metric; selection therefore falls back to p_{clean}. This score predicts exception free execution, whereas publication also requires a valid CSV and an accepted validation metric.

The denoising trials repeatedly encounter image shape errors and malformed submissions (missing identifiers, wrong row counts, or mismatched identifiers). Each produces one format valid candidate that is rejected by metric review. The two bird trials suffer repeated data parsing errors, including 178 recorded `ParserErrors`. In the Jigsaw trial, six timeouts consume 5.53 hours; its only exception free execution lacks a finite accepted metric and receives an HTTP 500 response from the submission validator. Thus the recorded failures include both program defects and a validation service error; reviewer rejection reasons are recorded assessments, not independently verified diagnoses.

Fusion publishes a verified submission on five of these six corresponding task–seed pairs; each first publication is a synthesized program. Fusion also fails on one pair where UCB succeeds (plant pathology, seed 35), giving its net advantage of four valid submissions. These records locate UCB's coverage deficit in obtaining and verifying an initial program. They do not isolate a causal effect of the uncertainty bonus: it changes the pick relative to metric greedy selection in only five of the 978 recorded decisions, and the policies follow different trajectories.

Where the gain lands. Figure 13 compares, task by task, how much of Base AIDE's execution is lost to crashes and timeouts with what the world model guided policies gain there. Base AIDE scores worst on the tasks where it wastes most of its execution (a), and those are the tasks where guidance raises the rate of valid submissions (b): six of the nine more wasteful tasks are those on which Base AIDE sometimes ends without a valid submission. Percentile gains are larger on the more wasteful half of the tasks ($+10.8$ versus $+1.2$ points without the gate, $+15.8$ versus $+5.5$ with it), but their rank correlation with waste is not established (c); the two least wasteful tasks leave little headroom, with Base AIDE already at the 96th and 100th percentile.

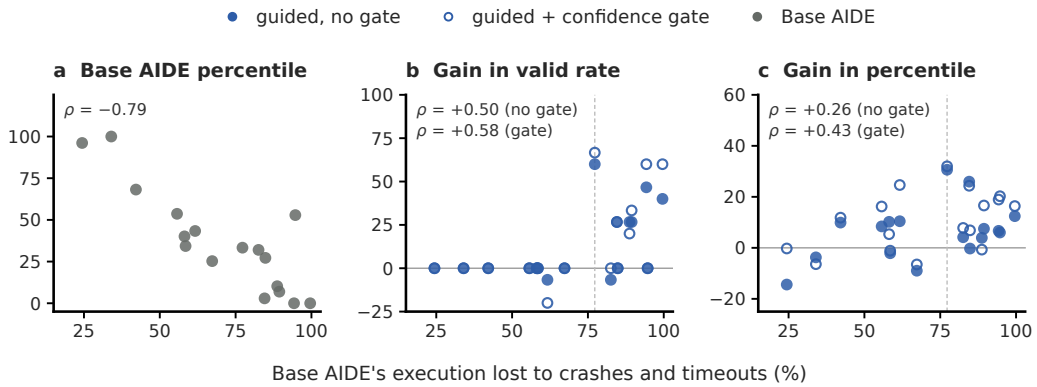


Figure 13: **Where the gain lands.** MLE-World's gains concentrate on the tasks where Base AIDE loses most of its execution to crashes and timeouts. Each point is one of the 17 tasks; x: share of Base AIDE's real execution time spent on runs that crash or time out (per trial, averaged over seeds). (a) Base AIDE's mean percentile. (b, c) Gain over Base AIDE in rate of valid submissions and mean percentile (points), averaged over the five world model guided policies, without (filled) and with (open) the confidence gate; the dashed line marks the median task. Spearman $\rho\pm$ SE: (a) -0.79 ± 0.17; (b) 0.50 ± 0.18 without and 0.58 ± 0.16 with the gate; (c) 0.26 ± 0.25 and 0.43 ± 0.19.

Table 20: **Diagnostics on fixed states.** Screening and Fusion improve validity and test percentile; the test percentile gains of Pairwise and Sequential from predicted terminal output are not established. Selection, Pairwise, and Fusion share their input pool; Sequential shares its starting candidate. *Uniform*: expected outcome of a random pick. *Screen*: random pick among candidates MLE-World predicts usable with a finite metric, else the largest $p_{\text{clean}}(x_s)$. + *ranking*: the screened candidate with the best predicted metric. *Real validation*: best real validation score, else the ranking's pick. Sequential uses the same selector by predicted metric and fallback in every condition; *Structured predictions* omit terminal output from revision feedback. Test pct. is zero if invalid. Δ: paired difference from the row above within each panel $\pm$ SE over 4,000 hierarchical task/run bootstrap resamples. †Executes all four inputs. ‡Uses real execution feedback during revision.

Rule	Mean percentile	Δ mean pct. (points)	Valid (%)	Δ valid (points)
Selection: pick one of the same four candidates				
Uniform over the pool	10.9	–	26.1	–
Screen by predicted outcome	16.4	+5.51 ±1.18	41.9	+15.8 ±3.1
+ ranking by predicted metric	17.1	+0.68 ±0.79	42.5	+0.5 ±1.3
Real validation of all four†	19.8	+2.65 ±2.19	47.1	+4.6 ±2.6
Pairwise: judge the same four candidates				
Code only	13.0	–	32.7	–
Predicted terminal output	15.6	+2.60 ±1.89	37.9	+5.2 ±3.6
Real terminal output†	14.6	−1.03 ±1.46	34.6	−3.3 ±2.2
Sequential: refine the same starting candidate				
No revision feedback	10.4	–	28.0	–
Structured predictions	12.3	+1.86 ±2.16	31.2	+3.2 ±4.5
Predicted terminal output	15.3	+2.99 ±2.50	38.0	+6.8 ±4.3
Real terminal output‡	15.0	−0.26 ±3.48	39.4	+1.4 ±4.9
Fusion: synthesize from the same four candidates				
Code only	14.0	–	36.7	–
Predicted terminal output	20.6	+6.55 ±2.87	49.2	+12.5 ±4.5
Real terminal output†	20.4	−0.21 ±1.71	46.5	−2.7 ±4.3

P DIAGNOSTICS ON FIXED STATES

Table 20 fixes the starting task state to isolate what MLE-World's predictions contribute to a local decision, separately from the rest of the search. We sample 153 states from recorded agent runs, three per task and stage (draft, debug, improve) on each of the 17 tasks, without consulting their outcomes. At each state, Qwen3.5-9B generates four candidates and MLE-World predicts their terminal outputs and structured fields. Selection, Pairwise, and Fusion use these same four candidates; Sequential shares the first candidate and generates three revisions under each feedback condition. We execute and grade every candidate and every program a rule produces; each rule's choice is recorded before any private test grade is read. We omit UCB because these local outcomes do not measure the value of exploration through later search decisions.

Model calls. The common pool contains four input programs. Pairwise adds three judge calls; Fusion adds one generation call to synthesize a fifth program. Sequential instead adds three revision calls to the shared first candidate and predicts all four attempts for final selection; it may also call a judge when all four are predicted unusable. These are the nominal calls before generation retries. Including code-parsing retries, each Fusion condition in Table 3 makes 153 synthesis calls over 150 states (1.02 per state). Pairwise makes exactly three judge calls per state. Predicted-feedback Pairwise and Fusion use four world-model predictions. Thus these comparisons share task states and inputs, but do not match total model calls or execution costs.

Screening and Fusion improve immediate outcomes. To isolate the contribution of predictions, we fix the task state and its four generated candidates, execute and grade every candidate, and vary

only how the predictions are used (Table 20). Screening and Fusion show the clearest gains in this controlled comparison. First, the predictions prevent failed runs: choosing among the candidates MLE-World predicts will run raises the share of chosen programs that yield a valid submission from 26.1% to 41.9% ($+15.8$ points), and predicted feedback raises fusion's share from 36.7% to 49.2% ($+12.5$ points). As in complete runs, the chosen program executes about as long (256 versus 260 seconds), but more of that time goes to runs that finish cleanly (135 versus 96 seconds; $+39$ seconds). Second, test percentile rises by 5.51 and 6.55 points, mostly through this higher validity. Subsequent metric ranking adds only 0.68 points after screening. Fusion's observed means with predicted and real feedback are 20.6 and 20.4 percentile.

Selection. Each rule picks one of the same four candidates. The screen picks uniformly among candidates predicted usable with a finite predicted metric and, when none qualifies, the candidate with the largest $p_{\text{clean}}(x_s)$. No candidate qualifies in 80 of the 153 states. In the 73 states where the screen keeps at least one candidate, it gains 9.16 percentile points; the fallback states gain 2.29 points. Ranking the screened candidates by predicted metric adds little. Selecting by the real validation scores of all four, which requires executing every candidate, reaches 19.8, 2.65 points above ranking by predicted metric.

Pairwise selection. The same four candidates are compared in pairs by a judge that receives code only, predicted terminal output, or real terminal output. Predicted terminal output raises test percentile by 2.60 points over code only, which is not established; validity rises from 32.7% to 37.9%.

Sequential refinement. Each condition starts with the same first candidate and generates three revisions, using no revision feedback, structured predictions without terminal output, predicted terminal output, or real terminal output. All conditions select among their four attempts by predicted metric with the same fallback rule, so the intervention changes revision feedback rather than final selection. Predicted terminal output raises test percentile by 4.85 points over no feedback and by 2.99 over structured predictions; neither gain is established.

Fusion. Fusion combines the same four candidates into a new program given code only, MLE-World's predicted feedback, or real feedback from executing all four. Predicted feedback raises the rate of valid submissions by 12.5 points and test percentile by 6.55, and real terminal output scores no higher (-0.21 points). Three states are excluded from every Sequential and Fusion condition because their runs did not complete (one exceeded the generator's context limit and two were stopped during execution); the exclusion uses completion status alone. Letting the three missing paired differences take any value from -100 to $+100$ bounds Fusion's gain from predicted feedback between $+4.52$ and $+8.44$ points.

Real terminal output does not establish a further local gain. Replacing predicted with real terminal output changes test percentile by -1.03 points for Pairwise (paired 95% interval $[-4.05, 1.80]$), -0.26 for Sequential ($[-7.28, 6.44]$), and -0.21 for Fusion ($[-3.45, 3.36]$). Selecting by real validation scores instead of predicted metrics gains $+2.65$ points, also uncertain ($[-0.24, 8.01]$). These comparisons find no established gain from substituting real feedback in the tested local decisions. They do not establish equivalence between predicted and real feedback or rule out benefits over longer searches.

Fusion leads the fixed state comparison. On the 150 states shared by all four local methods, Fusion has the highest mean percentile (20.6), followed by greedy selection by metric (17.3), Pairwise (15.4), and Sequential (15.3). Fusion exceeds Pairwise by 5.22 points (paired 95% interval $[1.56, 10.04]$) and Sequential by 5.31 ($[0.26, 11.77]$). Pairwise and Sequential differ by just 0.09 points ($[-4.31, 4.62]$). Fusion also exceeds greedy selection by metric in mean quality by 3.28 points, although this interval includes zero ($[-0.09, 8.30]$). These comparisons support Fusion's advantage over Pairwise and Sequential; the higher mean relative to metric selection is descriptive.

Relation to production BoN. The 19.8 percentile real validation reference in Table 20 additionally executes all four inputs, whereas production BoN executes only its selected program. The diagnostic's predicted metric rule is also not the full BoN policy: when no candidate qualifies, it uses the shared $p_{\text{clean}}(x_s)$ fallback. This occurs in 80 of the 153 pools. Production BoN instead

returns the first usable candidate or invokes the failed pool judge (Algorithm 1). Moreover, the diagnostic measures one local output at a fixed starting state, with different costs of model calls, rather than the final submission after eight hours of changing candidate pools and search states.

Where the eight hour BoN–Fusion gap appears. Fusion reaches 47.2 percentile versus BoN's 39.5 in Table 2. Both final validity (96.1% versus 88.2%) and mean percentile among valid submissions (49.1 versus 44.8) are higher; these conditional means use each policy's own valid trials (Table 19). Fusion also records more verified new bests per trial (16.5 versus 6.9) despite fewer committed search steps (59.8 versus 68.3), and a larger share of its real runs finish cleanly (64.5% versus 29.7%). These observations are consistent with more productive candidate generation and refinement. They do not identify which mechanism causes the final quality gap: attributing it to repeated synthesis or fallback behavior requires a matched intervention over complete searches.

Q CHOICE OF PRIMARY AGENT METRIC

MLE-bench compares each final submission with the competition's human leaderboard. Its medals use Kaggle's thresholds: gold requires the top 10% of teams on leaderboards with fewer than 100 teams, the top 10 on leaderboards with 100–249 teams, and the top 10 plus 0.2% of all teams (rounded down) otherwise. Ten of our 17 leaderboards have more than 1,000 teams, where gold means roughly the top 1%. Leaderboard percentile instead measures the share of entries a submission beats. We use percentile as the primary metric because medal rates are all or nothing: 12 of our 17 tasks never yield a gold, while one task with a small leaderboard yields more than half of all golds. Hambardzumyan et al. (2026) likewise rank agents by percentile because medal thresholds amplify noise and hide progress on hard tasks, and Foster et al. (2026) report a continuous normalized score. Tables 21 and 22 break down the 306 final submissions of the six 8 h ungated policies in Table 2 by task and by policy.

One task supplies more than half of all golds. Detecting-insults has a leaderboard of 50 teams, so gold means the top five. Seventeen of its 18 submissions win gold; only one Fusion seed misses. The task accounts for 17 of the 31 golds and adds up to three golds, or 5.9 points of gold rate, to every policy without separating them.

Most tasks never produce a medal. Twelve tasks yield no gold in any submission and nine yield no medal of any kind, so medal rates ignore them. Percentile still separates policies on these tasks. Means for each policy range from 0.0 to 36.6 on mlsp-2013-birds, where three policies never produce a valid submission, and from 2.9 to 53.9 on English text normalization, where a single submission earns a medal.

Where golds occur, a single seed decides them. On the other four gold tasks, gold requires the 98.7th to 99.1st percentile or the best leaderboard score. On aerial-cactus, 150 teams share a perfect AUC of 1.0, so gold requires exactly 1.0: a submission scoring 0.99996 ranks at the 87.7th percentile and wins no medal. With three seeds per task, one seed moves a policy's gold rate by 2.0 points.

Gold rate can reverse the ranking. Base AIDE has twice Fusion's gold rate (11.8% versus 5.9%; paired difference -5.9 points), yet Fusion submits valid solutions more often (96.1% versus 78.4%) and reaches a higher mean percentile ($+10.4$ points). Pairwise exceeds Fusion in gold rate ($+9.8$ points) but trails it in mean percentile (-4.4 points). Table 22 shows why. Fusion's three golds contribute 5.9 of its 47.2 percentile points, and its 46 other valid submissions, at a mean 45.8th percentile, contribute 41.3. Pairwise's eight golds contribute 15.4 of its 42.8 points, but its 39 other valid submissions average only the 35.8th percentile, and its four invalid submissions score zero.

Percentile has its own limit. Where leaderboard scores cluster, small score gains move percentile sharply: on English text normalization, accuracy 0.964 ranks at the 6th percentile and 0.990 at the 47th. Averaging over 17 equally weighted tasks limits the influence of any one leaderboard. We therefore base conclusions on mean percentile and report medal rates only for comparison with MLE-bench.

Table 21: **Medals by task.** More than half of all golds come from one task and twelve tasks yield none, yet policies still differ in percentile on tasks without medals. The 306 final submissions of the six 8 h ungated policies in Table 2 (18 per task: six policies, three seeds). Teams: leaderboard size. Gold rank: the rank a submission must reach for gold. Gold pct.: percentile of a submission scoring exactly the gold threshold. Golds and Any medal count submissions. Policy pct.: lowest and highest of the six mean percentiles of the policies. †Gold requires the best leaderboard score, which many teams share: AUC 1.0 on aerial-cactus (150 teams) and log loss 0.0 on leaf-classification (24 teams).

Task	Teams	Gold rank	Gold pct.	Golds	Any medal	Policy pct. (range)
Tasks with a gold						
`detecting-insults-in-social-commentary`	50	5	90.0	17	17	81.3–100.0
`dogs-vs-cats-redux-kernels-edition`	1,315	12	99.1	4	9	33.3–95.5
`plant-pathology-2020-fgvc7`	1,318	12	99.1	4	5	10.1–92.9
`tabular-playground-series-dec-2021`	1,189	12	98.7	4	4	46.3–72.7
`aerial-cactus-identification`	1,221	12	87.7†	2	2	50.4–69.8
Medals, but no gold						
`nomad2018-predict-transparent-conductors`	879	11	98.7	0	15	62.7–96.1
`random-acts-of-pizza`	462	10	97.8	0	14	68.2–87.7
`text-normalization-challenge-english-language`	261	10	96.2	0	1	2.9–53.9
No medal						
`spooky-author-identification`	1,242	12	99.0	0	0	40.1–55.4
`leaf-classification`	1,596	13	98.5†	0	0	17.8–47.9
`dog-breed-identification`	1,281	12	99.1	0	0	17.6–42.8
`tabular-playground-series-may-2022`	1,152	12	98.4	0	0	27.5–36.6
`mlsp-2013-birds`	81	8	90.1	0	0	0.0–36.6
`new-york-city-taxi-fare-prediction`	1,485	12	99.2	0	0	5.3–25.3
`denoising-dirty-documents`	162	10	93.8	0	0	0.0–22.2
`jigsaw-toxic-comment-classification-challenge`	4,539	19	99.6	0	0	7.0–20.8
`text-normalization-challenge-russian-language`	163	10	93.9	0	0	9.8–19.2
All 17 tasks				31	67	

Table 22: **Medals and percentile by policy.** Gold counts turn on a few submissions, whereas most of each policy's mean percentile comes from submissions without a gold. Counts over 51 trials per policy. Mean percentile (Total, as in Table 2) splits into points from gold medal trials and from other valid trials; invalid submissions contribute zero, and rounded components may not sum to the total.

	Golds				Mean percentile		
Policy	detecting-insults	Other	Any medal	Invalid	Gold trials	Other valid trials	Total
Base AIDE	3	3	11	11	11.8	25.1	36.9
Sequential	3	3	11	5	11.5	29.5	41.1
Pairwise	3	5	14	4	15.4	27.4	42.8
Fusion	2	1	11	2	5.9	41.3	47.2
BoN	3	0	9	6	5.9	33.6	39.5
UCB	3	2	11	6	9.8	35.3	45.1

R EXECUTION PREDICTION ON TASKS EXCLUDED FROM FINETUNING

Setup. We evaluate execution prediction on APTOS Blindness Detection, 3D Object Detection, and RANZCR Catheter/Line Classification, all excluded from finetuning. All methods use the same 300 programs: 100 per task, sampled uniformly without replacement within stage from each task's 200-program confirmation set (34 Draft, 33 Improve, 33 Debug). An audit of all 497,056 finetuning examples found neither these task identities nor exact program matches after whitespace normalization.

Table 23: **Exploratory prediction on three tasks excluded from finetuning.** Accuracy (%, ± program bootstrap SE) on 300 programs, 100 per task with balanced agent stages. The fields and scoring match Table 1; DeepSeek uses default reasoning effort here. Bold denotes the best value in each column.

Method	Terminal output	Exception class	Crash class	Validation metric
DeepSeek	42.1±2.8	**63.7**±2.8	45.4±3.2	21.3±5.8
Coder	21.1±2.2	30.3±2.6	18.9±2.4	31.9±6.8
MLE-World (4B, no CoT)	**53.3**±2.8	60.3±2.8	**56.3**±3.1	**44.7**±7.3

Potential transfer mechanisms. New tasks can reuse familiar execution operations: loading images, transforming tensors, and adapting pretrained models. Failures in these shared operations may remain predictable even when the data and prediction target change (Table 24). Task-specific geometry, labels, and metrics introduce less familiar behavior. Shared code therefore offers a plausible route to transfer, though training family ablations are needed to establish its role.

Results and limitations. The pooled and per-task results (Tables 23 and 25) suggest that execution prediction can transfer to tasks excluded from finetuning, with gains that vary across output fields. Because failed executions dominate the sample, the evidence mainly concerns recognizing execution failures on new tasks. Predicting successful training outcomes and timeouts remains less established. A broader, prospectively selected evaluation and training family ablations would help distinguish transfer through shared code from generalization across task families and software environments.

Table 24: **Shared operations that could support transfer.** The middle column names tasks present in finetuning. These are proposed mechanisms, not measured causal effects.

Held out task	Related training tasks	Shared structure and remaining shift
APTOS Blindness Detection	Plant Pathology 2020; Dog Breed	Image classification with pretrained visual encoders; retinal severity introduces ordinal labels and a new metric.
3D Object Detection	Dog Breed; Aerial Cactus	Shared image, array, and PyTorch components can expose familiar import and file errors; 3D geometry and detection remain new task specific computations.
RANZCR Catheter/Line	Plant Pathology 2020	Image preprocessing and visual prediction heads are shared; chest radiographs and multiple catheter/line labels change the target.

Table 25: **Per task prediction accuracy on the three selected tasks.** Accuracy (%, ± program bootstrap SE) using the same methods and metrics as Table 23. Each task has 100 programs: 34 Draft, 33 Improve, and 33 Debug. Gray † cells have fewer than ten metric references; SE is omitted when the metric is undefined in any bootstrap resample.

Method	Terminal output	Exception class	Crash class	Validation metric
APTOS Blindness Detection				
DeepSeek	49.3±4.9	**68.0**±4.5	**59.0**±5.6	16.7±8.6
Coder	18.3±3.8	32.0±4.6	15.3±4.2	16.7±9.1
MLE-World (4B, no CoT)	**53.0**±4.9	60.0±4.8	53.6±5.6	**50.0**±12.1
3D Object Detection				
DeepSeek	35.7±4.7	52.0±4.9	26.5±5.0	31.8±10.1
Coder	30.0±4.6	30.0±4.6	25.6±5.1	45.5±10.7
MLE-World (4B, no CoT)	**48.7**±5.1	**53.0**±5.1	**47.0**±5.8	**50.0**±10.9
RANZCR Catheter/Line				
DeepSeek	41.3±4.9	**71.0**±4.4	51.0±5.4	0.0†
Coder	15.0±3.5	29.0±4.6	15.7±4.0	**28.6**†
MLE-World (4B, no CoT)	**58.3**±4.7	68.0±4.6	**67.5**±5.0	14.3†